

Python Coding for AI

Ch1: Python의 직관적 이해

Ch2: Python OOP

Ch3: What is Numpy & Why Numpy?

Ch4: What is Pandas & Why Pandas?

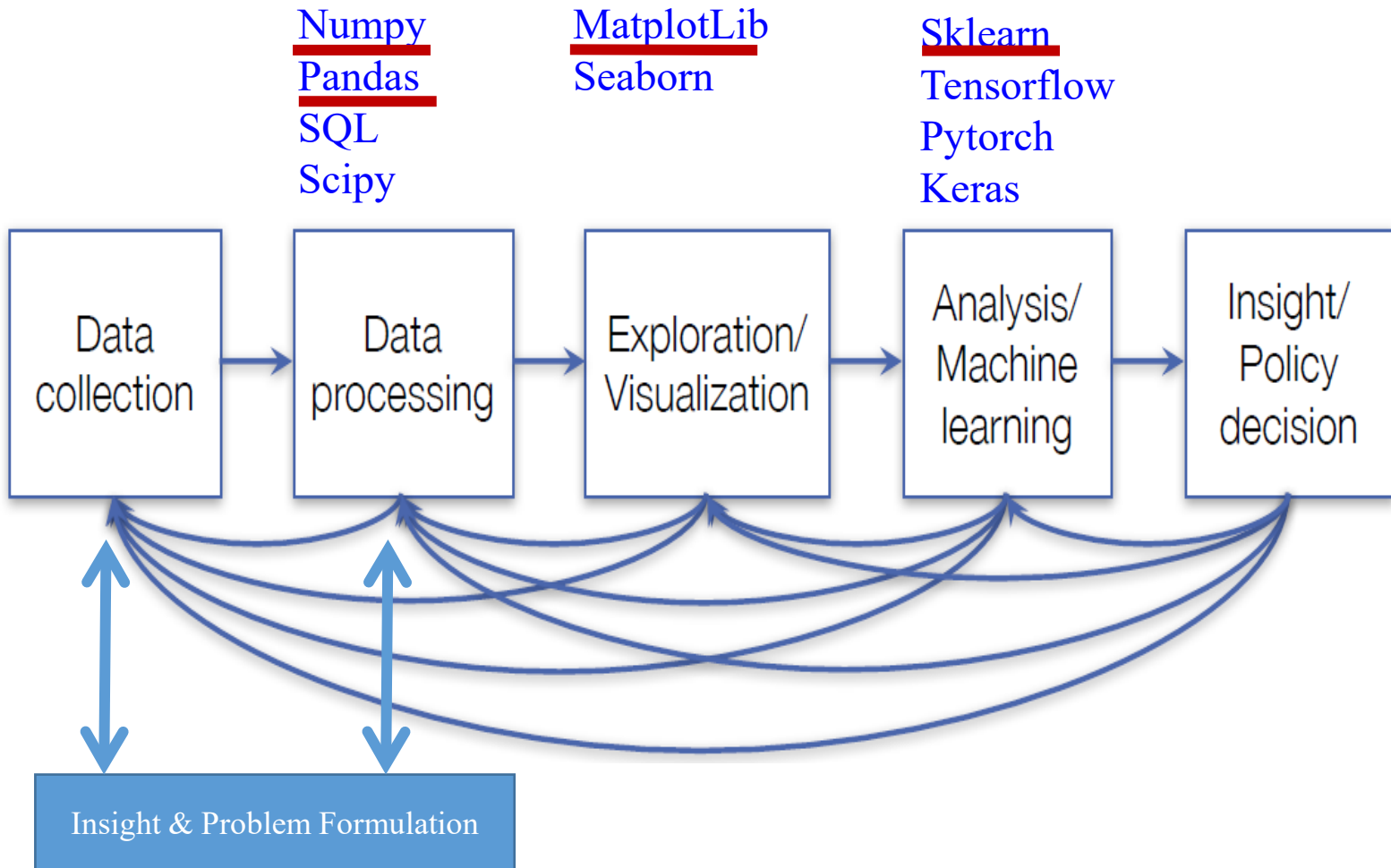
Ch5: What is Matplotlib & Why Matplotlib ?

Ch6: What is Sklearn & Why Sklearn ?

Ch7: What is Data Preprocessing & Why Data Preprocessing ?

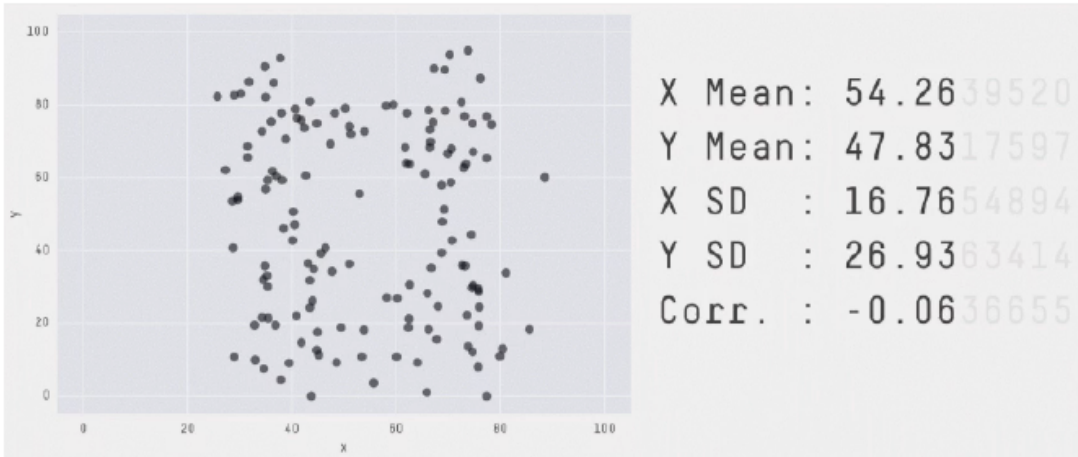
서울대 컴퓨터공학부
김형주 명예교수

Major Steps of Data Analytics



Visualization vs. Statistics

Visualization almost always presents a more informative (though less quantitative) view of your data than statistics (the noun, not the field)



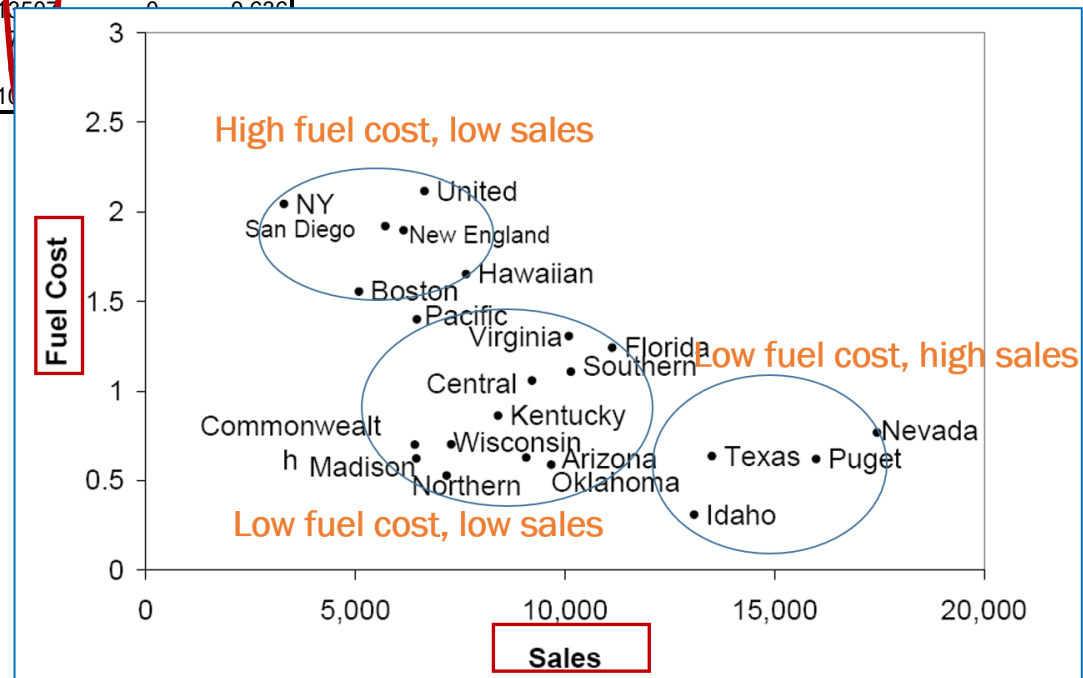
VS

6.58	65.2
6.42	78.9
7.19	61.1
7.00	45.8
7.15	54.2
6.43	58.7
6.01	66.6
6.17	96.1
5.63	100
6.00	85.9

<https://twitter.com/JustinMatejka/status/770682771656368128> Credit: @JustinMatejka, @albertocairo]

Power of Visualization

Company	Fixed_charge	RoR	Cost	Load	Δ Demand	Sales	Nuclear	Fuel_Cost
Arizona	1.06	9.2	151	54.4	1.6	9071	0	0.628
Boston	0.89	10.3	202	57.9	2.2	5088	25.3	1.555
Central	1.43	15.4	113	53	3.4	9212	0	1.058
Commonwealth	1.02	11.2	168	56	0.3	6423	34.3	0.7
Con Ed NY	1.49	8.8	192	51.2	1	3300	15.6	2.044
Florida	1.32	13.5	111	60	-2.2	1127	22.5	1.241
Hawaiian	1.22	12.2	175	67.6	2.2	7642	0	1.652
Idaho	1.1	9.2	245	57	3.3	3082	0	0.309
Kentucky	1.34	13	168	60.4	7.2	8406	0	0.862
Madison	1.12	12.4	197	53	2.7	6455	39.2	0.623
Nevada	0.75	7.5	173	51.5	6.5	7441	0	0.768
New England	1.13	10.9	178	62	3.7	6154	0	1.897
Northern	1.15	12.7	199	53.7	6.4	7179	50.2	0.527
Oklahoma	1.09	12	96	49.8	1.4	9673	0	0.588
Pacific	0.96	7.6	164	62.2	-0.1	6468	0.9	1.4
Puget	1.16	9.9	252	56	9.2	5991	0	0.62
San Diego	0.76	6.4	136	61.9	9	5714	8.3	1.92
Southern	1.05	12.6	150	56.7	2.7	10140	0	1.108
Texas	1.16	11.7	104	54	-2.1	12507	0	0.636
Wisconsin	1.2	11.8	148	59.9	3.5	7507	0	0.72
United	1.04	8.6	204	61	3.5	9071	0	2.044
Virginia	1.07	9.3	174	54.3	5.9	10140	0	1.108

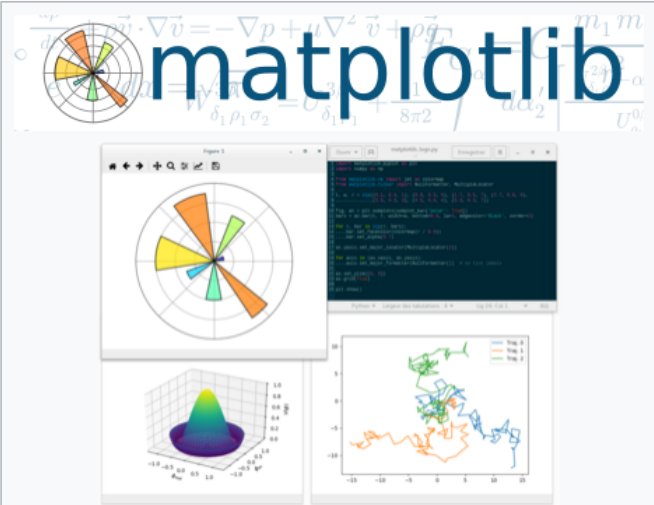


What is “matplotlib” module?

- Plotting library for Python
 - Reliant on NumPy
- MATLAB-style plotting interface with hierarchically organized elements
 - **pyplot** module: provides state-machine environment at the top of hierarchy
- Open Source (<http://matplotlib.org/>)
 - Original Author: **John D. Hunter**, 2003
→ **Michael Droettboom**, 2012
- pyplot tutorials
 - https://matplotlib.org/users/pyplot_tutorial.html

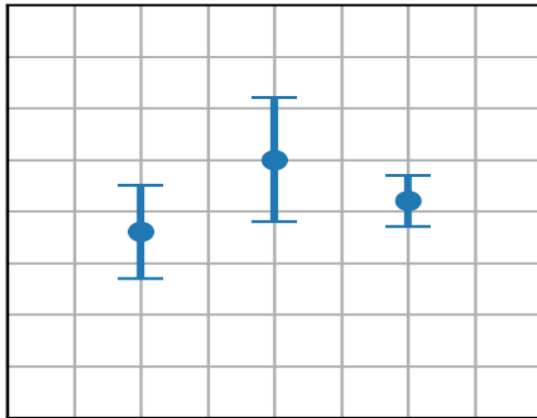


Matplotlib



Screenshot of Matplotlib plots and code

Original author(s)	John D. Hunter
Developer(s)	Michael Droettboom, <i>et al.</i>
Initial release	<u>2003; 20 years ago^[1]</u>
Stable release	<u>3.8.0^[2] / 15 September 2023;</u> 40 days ago
Repository	github.com/matplotlib/matplotlib
Written in	Python
Engine	Cairo, Anti-Grain Geometry
Operating system	Cross-platform
Type	Plotting
License	Matplotlib license
Website	matplotlib.org



errorbar(x, y, yerr, xerr)

Matplotlib: Visualization with Python

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.

- Create publication quality plots.
- Make interactive figures that can zoom, pan, update.
- Customize visual style and layout.
- Export to many file formats.
- Embed in JupyterLab and Graphical User Interfaces.
- Use a rich array of third-party packages built on Matplotlib.

Try Matplotlib (on Binder)



Getting Started



Examples



Reference

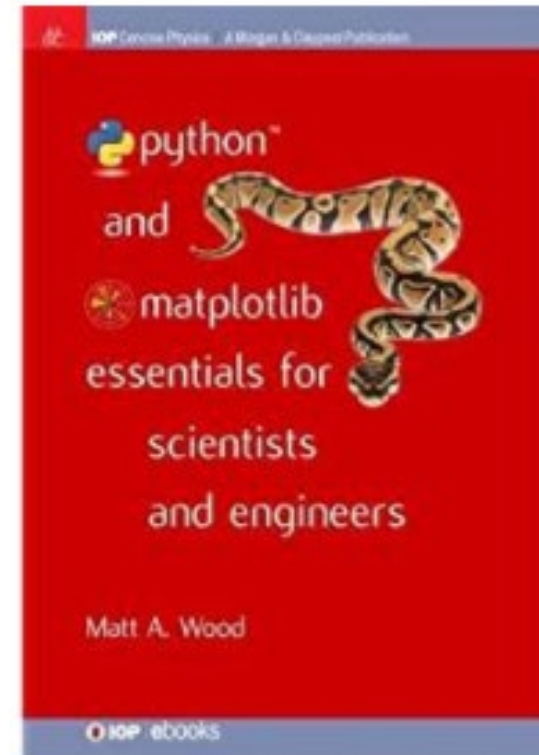
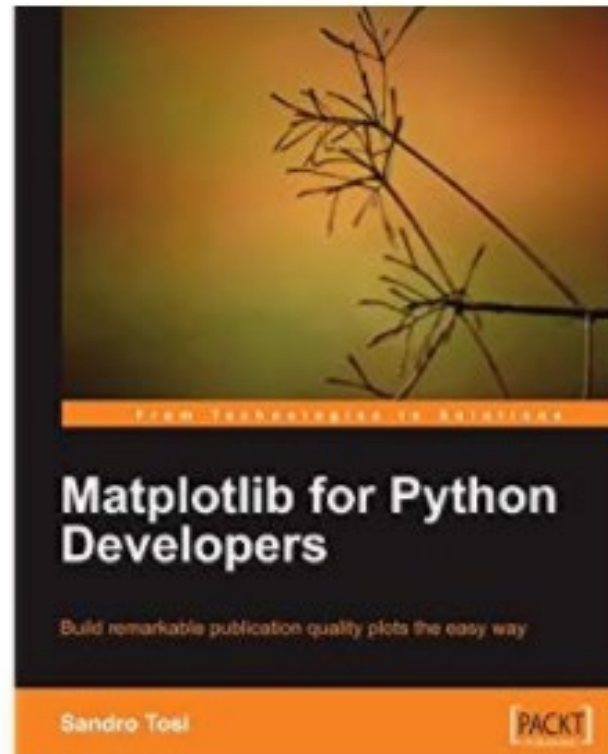
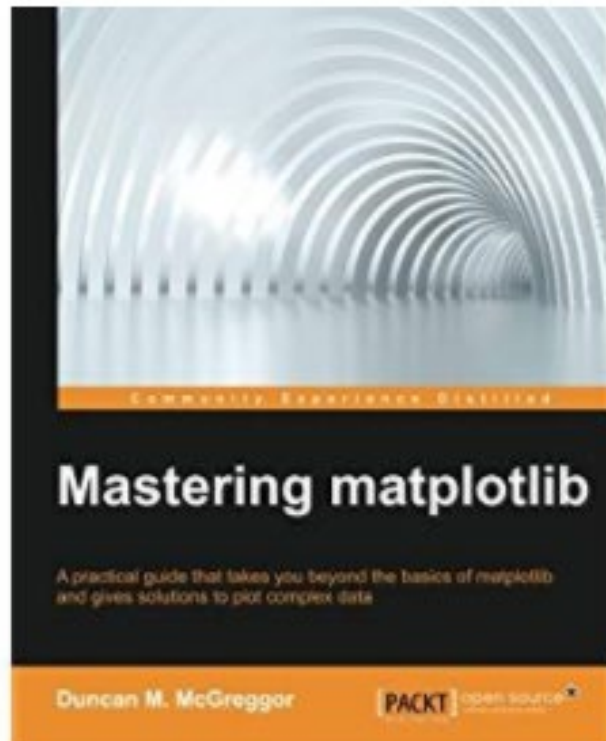


Cheat Sheets



Documentation

Many Matplotlib Books



Python Modules for Various Drawings, Diagrams, Data Plotting

- Tkinter Module : Canvas Drawing and GUI coding
- Django Module : Building Web-based GUI
- GraphViz Module : Drawing Various Trees and Graphs
- NetworkX Module : Graphviz + Graph processing
- Matplotlib and Seaborn Module : Various Data Plotting
- Plotly Module: Data Plotting, Widget Interaction, Data Animation
- Altair Module : Declarative Visualization, Interactive Charts

matplotlib.bezier
matplotlib.blocking_input
matplotlib.category
matplotlib.cbook
matplotlib.cm
matplotlib.collections
matplotlib.colorbar
matplotlib.colors
matplotlib.container
matplotlib.contour
matplotlib.dates
matplotlib.docstring
matplotlib.dviread
matplotlib.figure
matplotlib.font_manager
matplotlib.fontconfig_pattern
matplotlib.gridspec
matplotlib.image

matplotlib.legend
matplotlib.legend_handler
matplotlib.lines
matplotlib.markers
matplotlib.mathtext
matplotlib.mlab
matplotlib.offsetbox
matplotlib.patches
matplotlib.path
matplotlib.patheffects
matplotlib.projections
matplotlib.projections.polar
[matplotlib.pyplot](#)
matplotlib.quiver
matplotlib.rcsetup
matplotlib.sankey
matplotlib.scale
matplotlib.sphinxext.plot_directive

- Within pyplot submodule

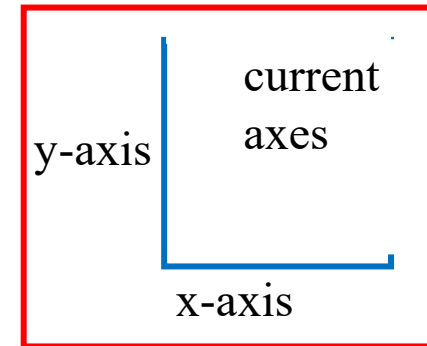
- plot()
- scatter()
- bar()
- hist()
- pie()
- boxplot()
- subplot()

matplotlib.style
matplotlib.table
matplotlib.testing
matplotlib.testing.compare
matplotlib.testing.decorators
matplotlib.testing.exceptions
matplotlib.texmanager
matplotlib.text
matplotlib.textpath
matplotlib.ticker
matplotlib.tight_bbox
matplotlib.tight_layout
matplotlib.transforms
matplotlib.tri
matplotlib.type1font
matplotlib.units
matplotlib.widgets

“matplotlib” module: pyplot submodule

- Import library: `import matplotlib.pyplot as plt`

- Pyplot uses 1 Figure (canvas, 도화지)
with 1 Axes (좌표평면) in default
- Each objects can be added as needed



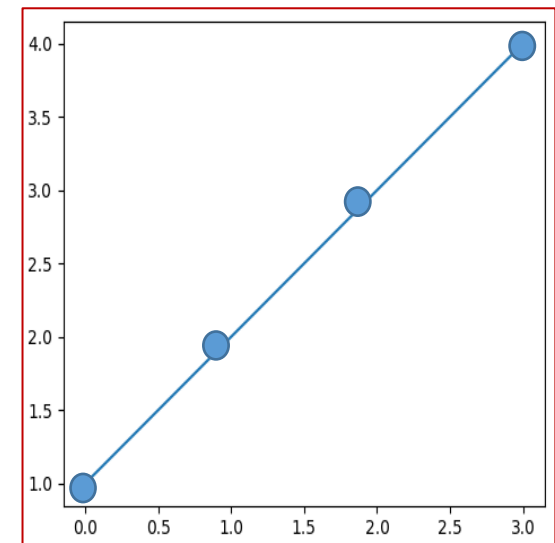
- `plt.plot()` on current axes

* `plt.plot( [0, 1, 2, 3], [1, 2, 3, 4] )`

x축의 값들, y축 값들

* `plt.plot( [1, 2, 3, 4] )`

x축의 data공급이 없으면,
x축값은 1의 간격으로 0 부터 유추



Pyplot submodule functions

- `plot()` in PyPlot
- `scatter( )` in PyPlot
- `bar()` in PyPlot
- `pie( )` in PyPlot
- `hist()` in PyPlot
- `boxplot()` in PyPlot
- `figure( )` & `subplot()` in PyPlot

- Matplotlib 작업
 - submodule pyplot 에 있는 함수 실행
 - 그림 그리는 대상 Object (Pyplot Function들의 input)
 - Python list or Numpy ndarray or Pandas Object

matplotlib.pyplot.plot() = plt.plot()

plt.plot(**args*, *scalex=True*, *scaley=True*, *data=None*, ****kwargs** **)**

x: x좌표, array-like or scalar

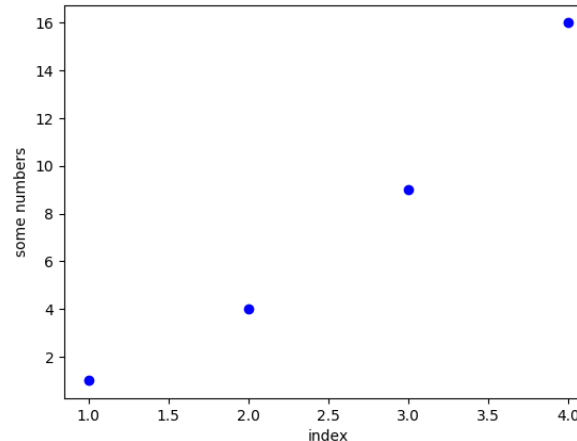
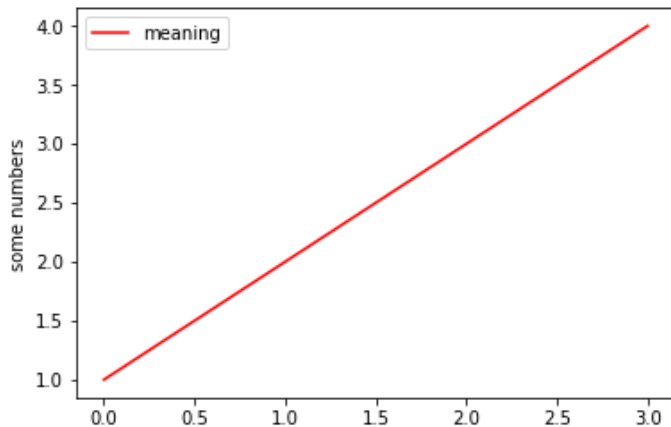
y: y좌표, array-like or scalar

c (color) : line graph나 scatter plot 의 색을 지정

data: x, y, c 등의 값을 tuple로 묶어서 공급

marker: 점을 그릴때 점으로 사용되는 symbol

label: line graph 나 scatter plot 의 이름을 지정



plt.plot() : Line Graph (꺾은선 그래프) 그리기 [1/2]

- `plot(x, y)` # plot x and y using default line style and color
- `plot(x, y, 'bo')` # plot x and y using blue circle markers

```
import matplotlib.pyplot as plt
```

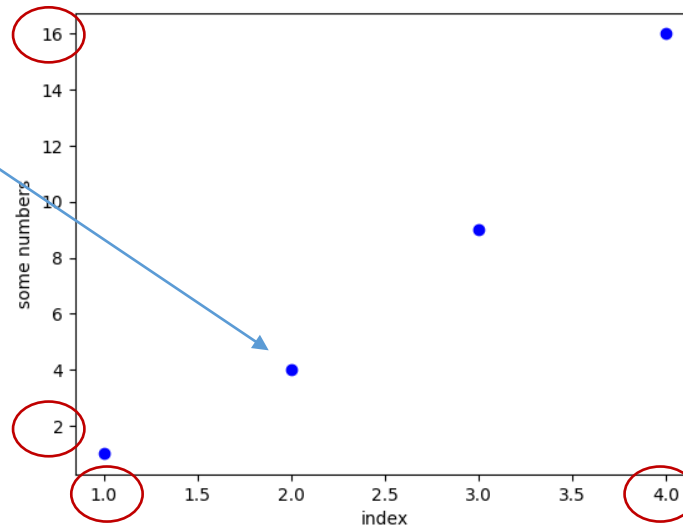
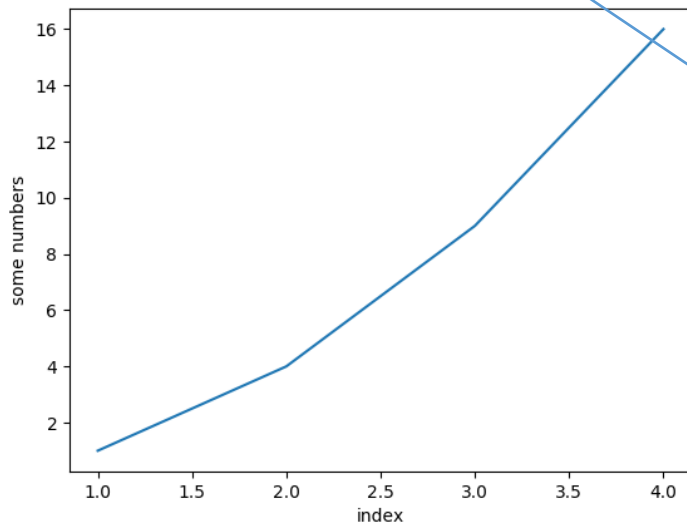
```
x = [1, 2, 3, 4]  
y = [1, 4, 9, 16]
```

```
plt.plot(x, y)  
plt.plot(x, y, 'bo')
```

Python list로 X, Y 값을 주고 있지만,
numpy array로 공급해도 same result

```
x = np.array([1, 2, 3, 4])  
y = np.array([1, 4, 9, 16])
```

marker가 없으면 blue line으로
꺾은선 그래프 (line graph)



Scatter plot (산점도) 도
plt.plot()으로 그릴수 있다

plt.plot() : Line Graph (꺾은선 그래프) 그리기 [2/2]

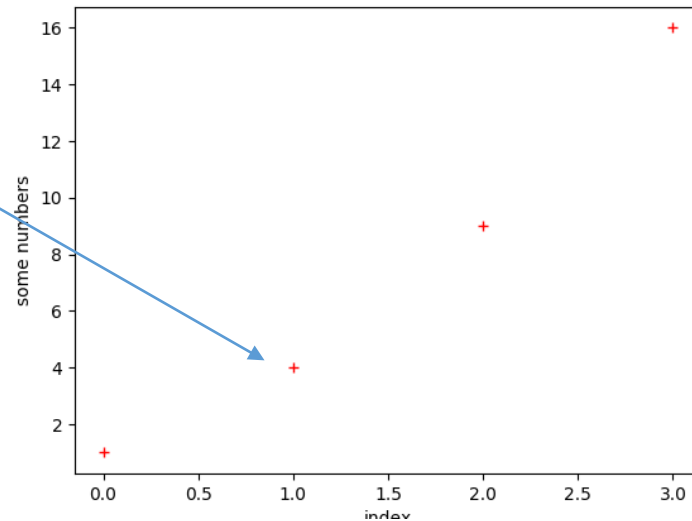
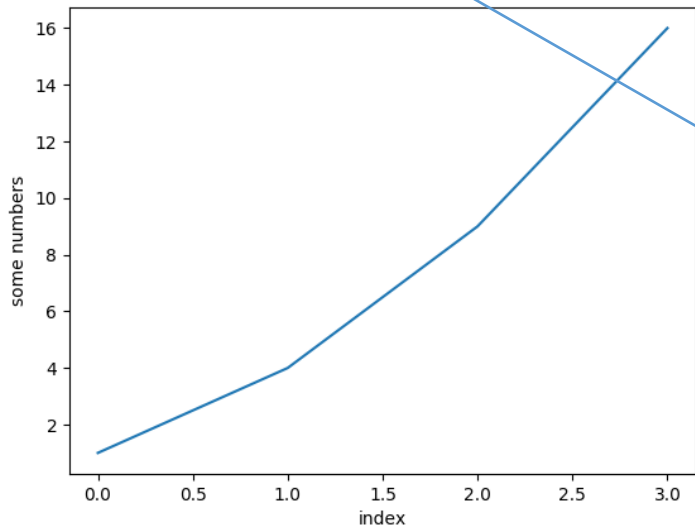
- `plot(y)` # plot y using x as index array 0..N-1
- `plot(y, 'r+')` # ditto, but with red plusses

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]  
y = [1, 4, 9, 16]
```

```
plt.plot(y)  
plt.plot(y, 'r+')
```

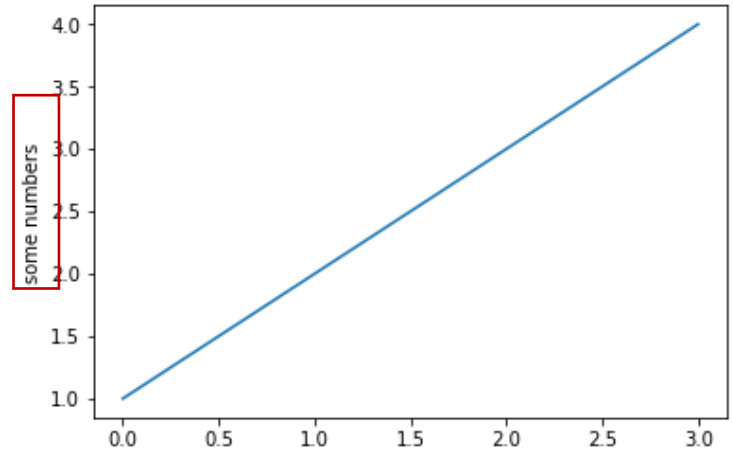
X축의 data공급이 없으면, X축값은
1의 간격으로 0 ~ y축 data개수



plt.ylabel(), plt.legend() 사용예

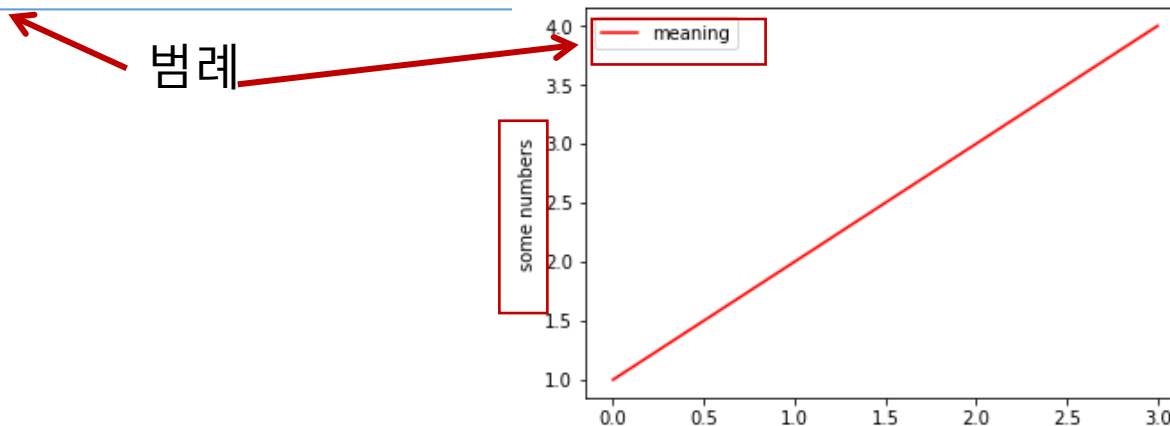
```
import matplotlib.pyplot as plt

rline = plt.plot([1, 2, 3, 4])
plt.ylabel('some numbers')
```



```
import matplotlib.pyplot as plt

rline = plt.plot([1, 2, 3, 4], color = 'red' , label = 'meaning')
plt.ylabel('some numbers')
plt.legend()
```



범례

plt.plot() & plt.axis() 사용예

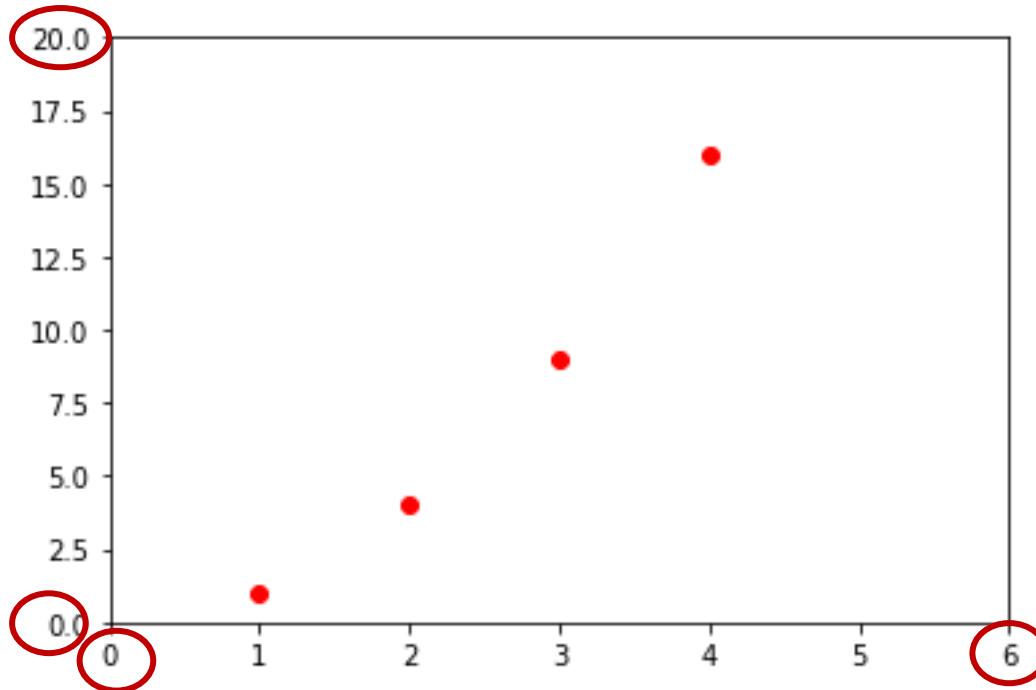
Change color & line style

```
%matplotlib inline
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
plt.axis([0, 6, 0, 20])
```

Setting Range of Axes:

[x축 시작점, x축 끝점, y축 시작점, y축 끝점]



- X축과 y축의 눈금(tick)의 크기는 점들의 분포를 보고 plt가 적절히 추론

Markers in plt.plot()

Ex: 'ro' 'b+' 'gD'



Color

character	color
<u>'b'</u>	blue
'g'	green
<u>'r'</u>	red
'c'	cyan
'm'	magenta
'y'	yellow
<u>'k'</u>	black
'w'	white

Line or Marker Style

character	description
'-'	solid line style
<u>'--'</u>	dashed line style
'-.'	dash-dot line style
':'	dotted line style
'.'	point marker
','	pixel marker
<u>'o'</u>	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker

'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
<u>'*'</u>	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
<u>'+'</u>	plus marker

plt.plot(): Multi-Line Graph 그리기

Try: `t = range(0., 5., 0.2)`

```
import numpy as np
import matplotlib.pyplot as plt
```

evenly sampled time at 200ms intervals

```
t = np.arange(0., 5., 0.2) ← array([0.0, 0.2, 0.4, ..., 4.6, 4.8])
```

```
plt.title("Drawing Multi-Line Graphs")
```

```
plt.plot(t, t, label="linear", color='red')
```

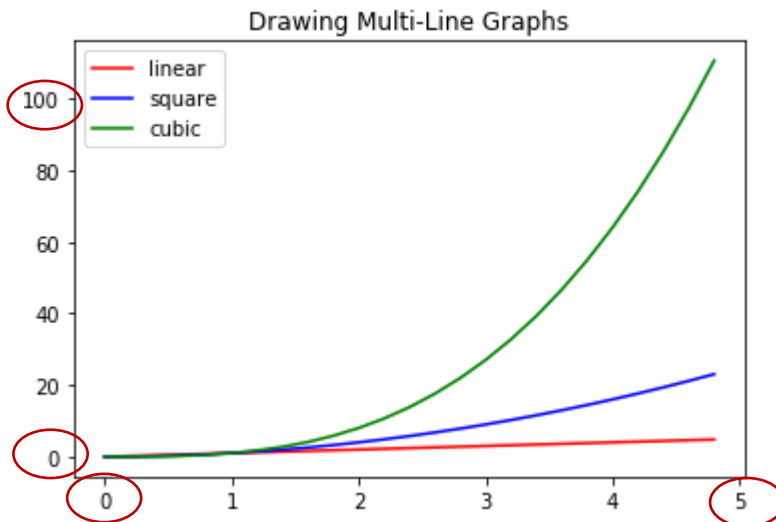
```
plt.plot(t, t ** 2, label="square", color='blue')
```

```
plt.plot(t, t ** 3, label="cubic", color='green')
```

```
plt.legend( ) ← 범례
```

graph의 label

$y = x$
 $y = x ** 2$
 $y = x ** 3$



- Plotting 3 different data at once
- Save plots with **plt.savefig**("file name")

$t**3$ 의 최대값이 125이므로
 y 축을 0 ~ 125정도로 표현!

plt.plot(): Multi Scatter Plots 그리기 [1/2] self study

```
import numpy as np
import matplotlib.pyplot as plt
```

Try: `t = range(0., 5., 0.2)`

```
# evenly sampled time at 200ms intervals
```

```
t = np.arange(0., 5., 0.2) ← array([0.0, 0.2, 0.4, ..., 4.6, 4.8])
```

```
# red dashes, blue squares and green triangles
```

```
plt.plot(t, t, 'r--', t, t ** 2, 'bs', t, t ** 3, 'g^')
```

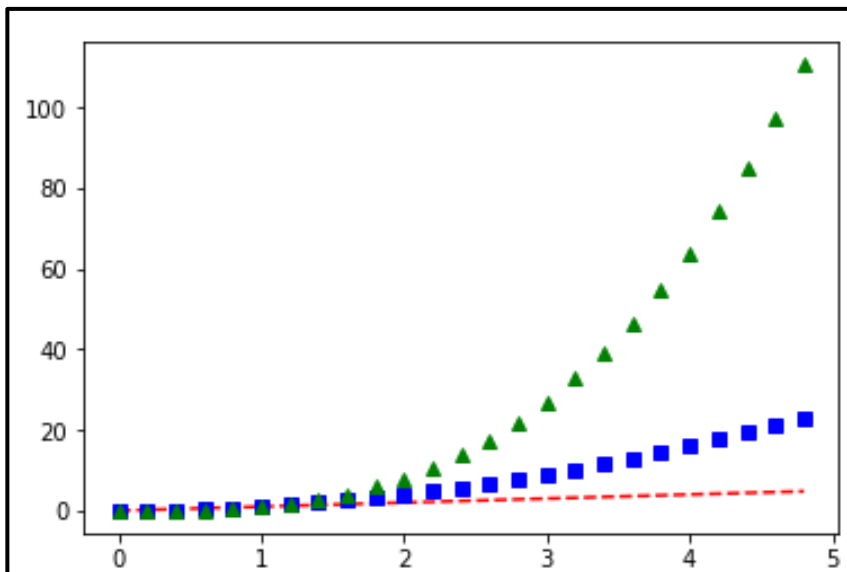
$y = x$
 $y = x ** 2$
 $y = x ** 3$

red dashed

blue square

green triangle

$y = x$
 $y = x ** 2$
 $y = x ** 3$



간편하게 Multi Scatter Plot을
그릴수 있지만, Legend 같은것
을 추가할수가 없다

plt.plot(): Multi Scatter Plots 그리기

[2/2]

self study

```
import numpy as np
import matplotlib.pyplot as plt
```

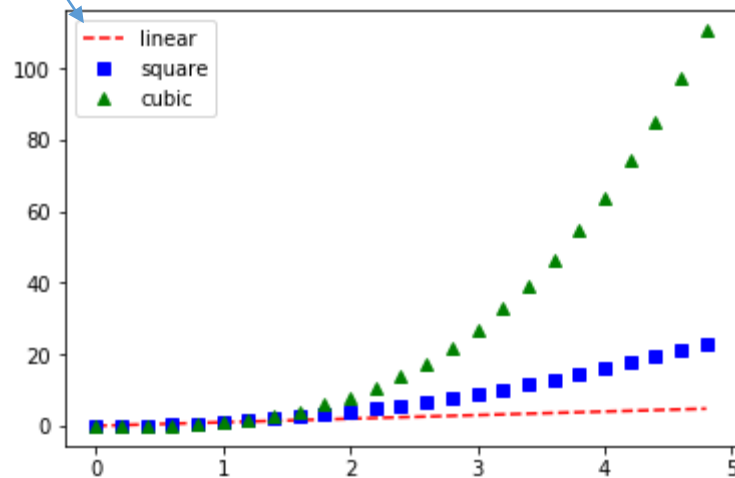
Try: `t = range(0., 5., 0.2)`

```
# evenly sampled time at 200ms intervals
```

```
t = np.arange(0., 5., 0.2) ← array([0.0, 0.2, 0.4, ..., 4.6, 4.8])
```

```
# red dashes, blue squares and green triangles
```

```
plt.plot(t, t, 'r--', label="linear") # red dashed
plt.plot(t, t ** 2, 'bs', label="square") # blue square
plt.plot(t, t ** 3, 'g^', label="cubic") # green triangle
plt.legend()
```

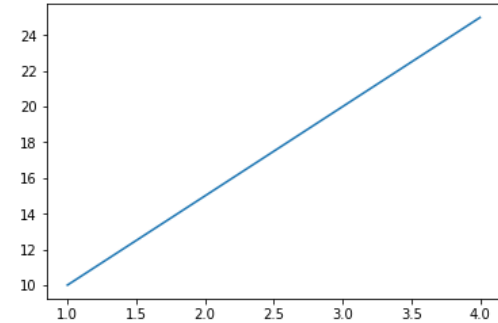


```
y = x
y = x ** 2
y = x ** 3
```

tick (눈금) 조절하기

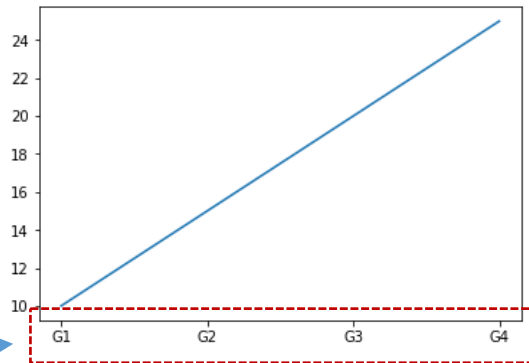
```
x = np.array([1, 2, 3, 4])  
y = np.array([10, 15, 20, 25])  
graph = plt.plot(x, y)
```

- default tick : x 축의 tick 0.5 each



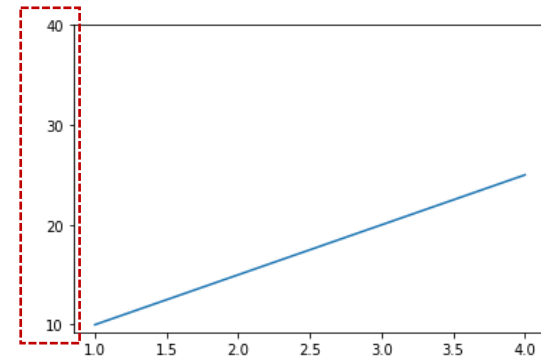
```
x = np.array([1, 2, 3, 4])  
y = np.array([10, 15, 20, 25])  
graph = plt.plot(x, y)  
plt.xticks(x, ('G1', 'G2', 'G3', 'G4'))
```

- tick 을 문자로 지정하기



```
x = np.array([1, 2, 3, 4])  
y = np.array([10, 15, 20, 25])  
graph = plt.plot(x, y)  
plt.yticks([10, 20, 30, 40])
```

- tick의 크기를 지정하기



Pyplot submodule functions

- plot() in PyPlot
- scatter() in PyPlot
- bar() in PyPlot
- pie() in PyPlot
- hist() in PyPlot
- boxplot() in PyPlot
- figure() & subplot() in PyPlot

- Matplotlib 작업
 - submodule pyplot 에 있는 함수 실행
 - 그림 그리는 대상 Object (Pyplot Function들의 input)
 - Python list or Numpy ndarray or Pandas Object

plt.scatter() : Scatter Plot (Bubble Chart)

- `matplotlib.pyplot.scatter( x, y, s, c, marker, alpha, ...)`

x, y: data positions

s: marker size

c: marker color { b }

marker: marker style { o }

Alpha: 0~1 값으로 투명도

- `plt.plot( )` 과 유사하지만 marker들의 크기와 색을 다양하게 Control 할수 있다



```
import numpy as np
import matplotlib.pyplot as plt
```

```
N = 20
```

```
x = np.random.rand(N)
```

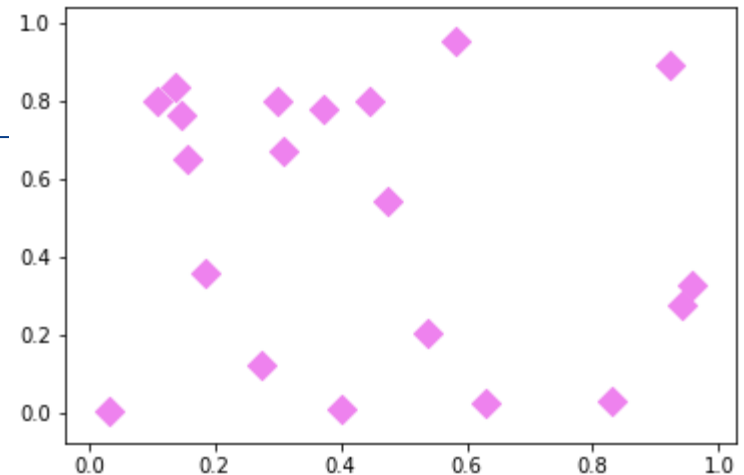
```
y = np.random.rand(N)
```

```
plt.scatter(x, y, s=100, c='violet', marker='D')
```

size

Color format string

diamond



`np.random.random() == np.random.rand()`

- Uniform distribution [0,1) 에서 20개의 random number

matplotlib-2.py 실행

RGB Color Table : # RR GG BB

- color format string

0 ~ 256

```
plt.scatter(x, y, s=100, c='violet', marker='D')
```

Basic colors:

Color	HTML /	Hex Code #RRGGBB	Decimal Code (R,G,B)
	Black	#000000	(0,0,0)
	White	#FFFFFF	(255,255,255)
	Red	#FF0000	(255,0,0)
	Lime	#00FF00	(0,255,0)
	Blue	#0000FF	(0,0,255)
	Yellow	#FFFF00	(255,255,0)
	Cyan	#00FFFF	(0,255,255)
	Magenta	#FF00FF	(255,0,255)
	Silver	#C0C0C0	(192,192,192)
	Gray	#808080	(128,128,128)
	Maroon	#800000	(128,0,0)
	Olive	#808000	(128,128,0)
	Green	#008000	(0,128,0)
	Purple	#800080	(128,0,128)
	Teal	#008080	(0,128,128)
	Navy	#000080	(0,0,128)

You can also use any locally defined standard color names.

```
>>> plt.scatter( [1], [2], c = 'Red' )
```

```
>>> plt.scatter( [1], [2], c = '#FF0000' )
```

상식적인 수준의 color 표현은 acceptable

예: violet, Violet, VIOLET

orange, Orange, ORANGE

- 두함수 다 point들을 그리는데 사용한다
- Plot() 의 본래 목적은 line graph을 그리는것!
- Scatter()에서는 marker들의 크기와 색을 다양하게 control 가능

```
np.random.seed(19680801)
```

```
N = 20
x = np.random.rand(N)
y = np.random.rand(N)
```

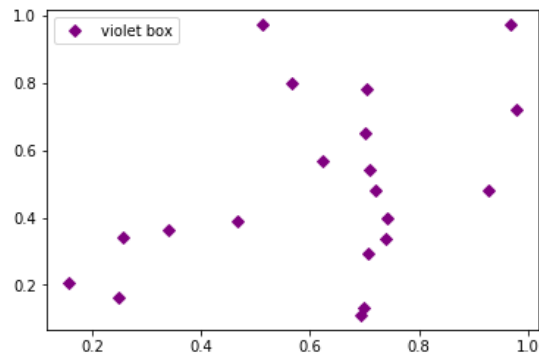
```
plt.plot( x, y, 'D ', c='purple', markersize=5, label='violet box' )
plt.legend()
```

```
np.random.seed(19680801)
```

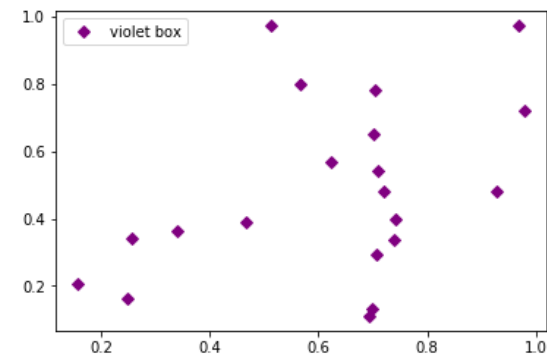
```
N = 20
```

```
x = np.random.rand(N)
y = np.random.rand(N)
```

```
plt.scatter( x, y, s=25, c='purple', marker='D', label='violet box' )
plt.legend()
```



같은크기



plt.scatter() : 다양한 Marker로 산점도 그리기

Scatter()에서는 marker들의 크기와 색을 다양하게 control 가능

```
import numpy as np
import matplotlib.pyplot as plt
```

```
N = 50
x = np.random.rand(N)
y = np.random.rand(N)
colors = np.random.rand(N)
area = (30 * np.random.rand(N)) ** 2
```

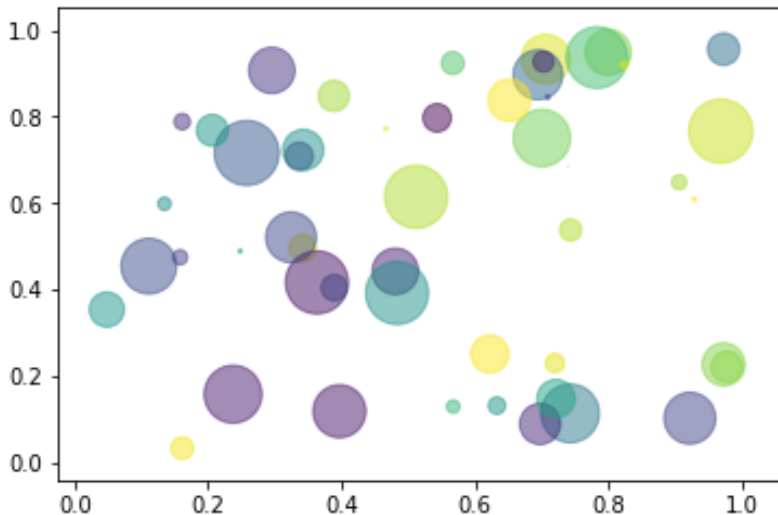
- Single color format string
- A sequence of n numbers to be mapped to cmap

← 각 point의 marker 크기 임의의 생성

```
plt.scatter(x, y, s=area, c=colors, alpha=0.5)
```

← 각 marker의 투명도 지정

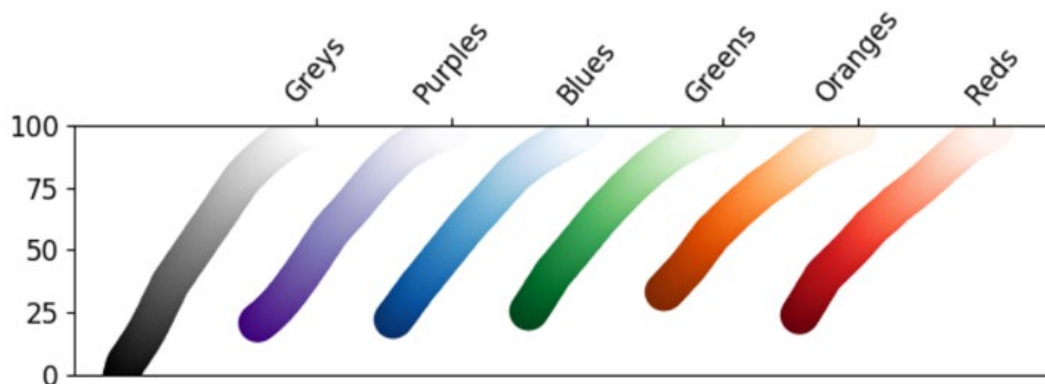
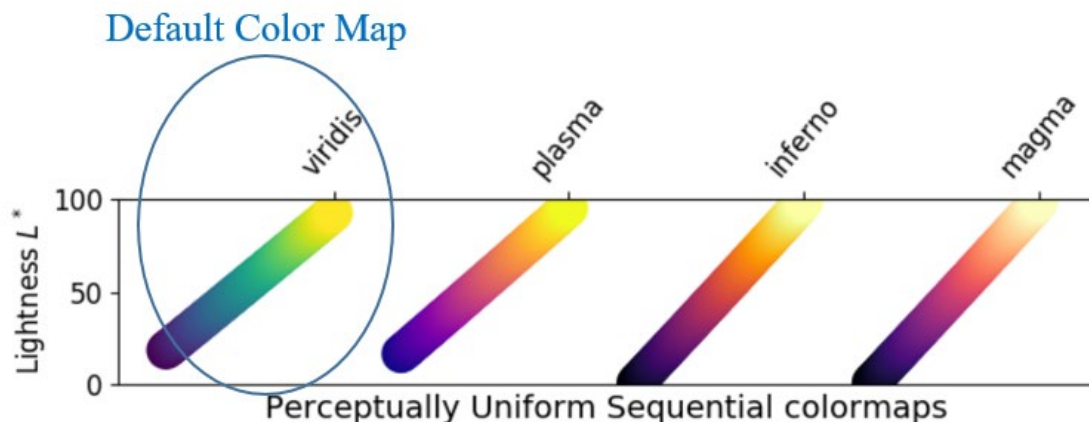
x좌표 수, y좌표 수, marker의 크기의 갯수, marker의 color의 수: 전부 같아야 함!



Matplotlib built-in colormaps

- 표현하려는 대상물이 N개라면 cmap에 있는 color중에서 대비가 잘 되도록 자동적으로 선택

`plt.scatter( ..... , cmap = 'plasma' )`



Default Color Map in plt.scatter()

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.arange(3)
y = np.arange(3)
t = np.arange(3)
```

array([0, 1, 2])

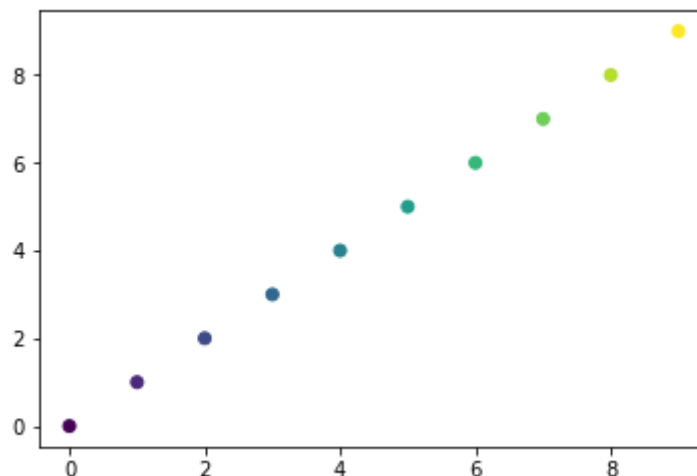
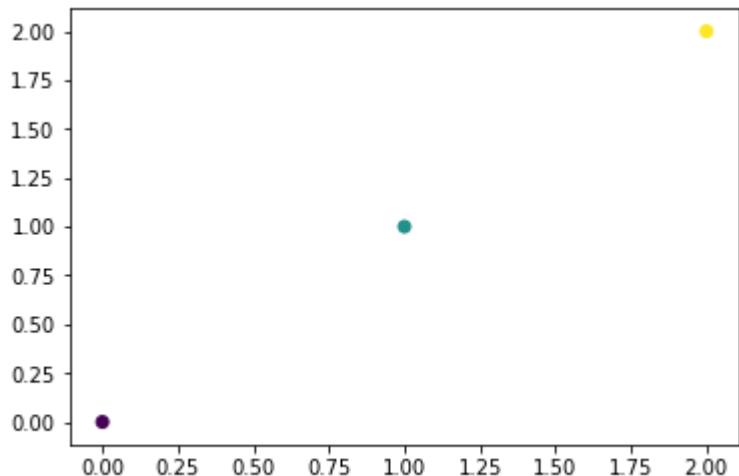
```
plt.scatter(x, y, c = t)
```

N개의 숫자를 주면
cmap에 있는 color
들을 대비가 잘 되
도록 적절하게 사용

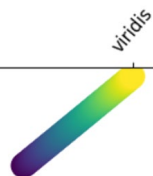
```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.arange(10)
y = np.arange(10)
t = np.arange(10)
```

```
plt.scatter(x, y, c = t)
```



Default Color Map



t = 0 → Violet
t = 1 → Green
t = 2 → Yellow

Pyplot submodule functions

- plot() in PyPlot
- scatter() in PyPlot
- bar() in PyPlot
- pie() in PyPlot
- hist() in PyPlot
- boxplot() in PyPlot
- figure() & subplot() in PyPlot

- Matplotlib 작업
 - submodule pyplot 에 있는 함수 실행
 - 그림 그리는 대상 Object (Pyplot Function들의 input)
 - Python list or Numpy ndarray or Pandas Object

plt.bar() : Plotting Bar Chart

[1/3]

- matplotlib.pyplot.bar(left, height, width, bottom, ...)

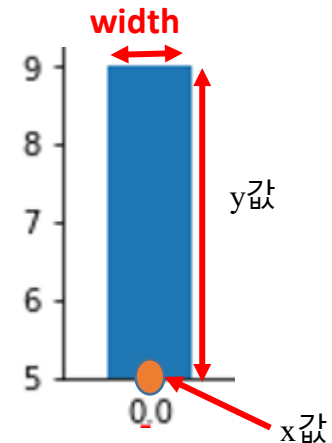
left: position of bar on x axis

height: height of bar

width: width of bar

bottom: baseline of y axis

Returns `barcontainer`



Out[14]: <Container object of 4 artists>

```
import numpy as np
import matplotlib.pyplot as plt

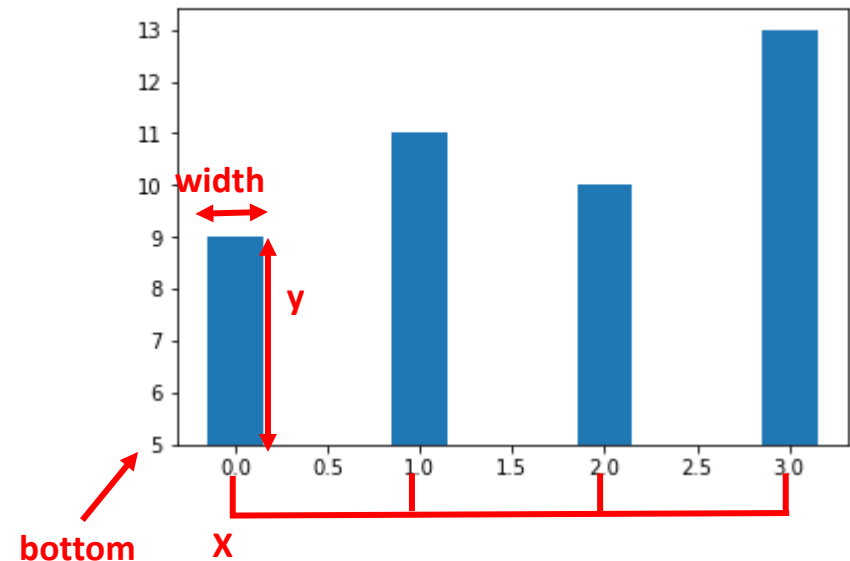
x = np.arange(4)  ← array([0, 1, 2, 3])
y = [4, 6, 5, 8]

plt.bar(x, y, width=0.3, bottom=5)
```

Positions of
bar on X axis

Height of bar

그림에서 bar chart
들의 시작점



[2/3]

plt.bar() : Plotting Side-by-Side Bar Chart

```
import numpy as np
import matplotlib.pyplot as plt

N = 5
menMeans = (20, 35, 30, 35, 27)
womenMeans = (25, 32, 34, 20, 27)

x = np.arange(N)  ← array([0,1,2,3,4])
width = 0.35

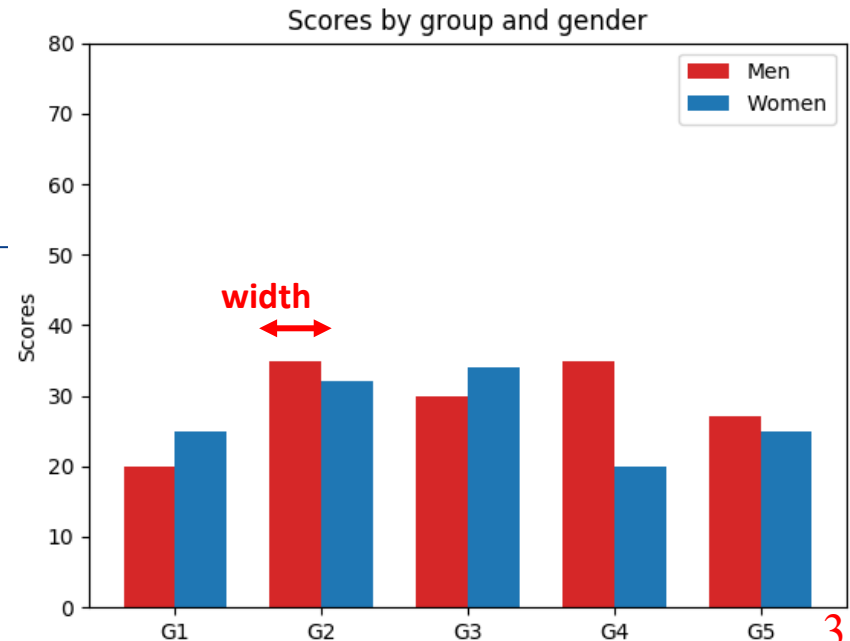
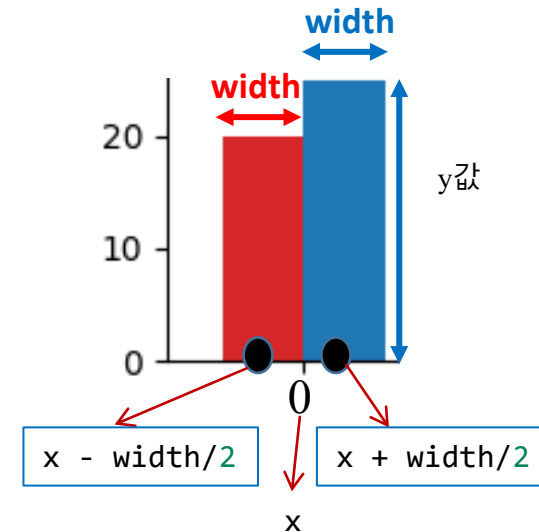
p1 = plt.bar(x - width / 2, menMeans, width, color= 'red' , label="Men")
p2 = plt.bar(x + width / 2, womenMeans, width, label="Women")
      ← default color: blue

plt.ylabel('Scores')
plt.title('Scores by group and gender')

plt.xticks(x, ('G1', 'G2', 'G3', 'G4', 'G5'))
plt.yticks(np.arange(0, 81, 10))  ← array([0,10,20,...,80])

plt.legend()
```

bar object들의 label들을
가지고 legend를 생성



바탕화면에 matplot-6.py로 저장

plt.bar() : Plotting Stacked Bar Chart

[3/3]

```
import numpy as np
import matplotlib.pyplot as plt
```

```
N = 5
menMeans = (20, 35, 30, 35, 27)
womenMeans = (25, 32, 34, 20, 27)
```

```
x = np.arange(N)
width = 0.35
```

← array([0,1,2,3,4])

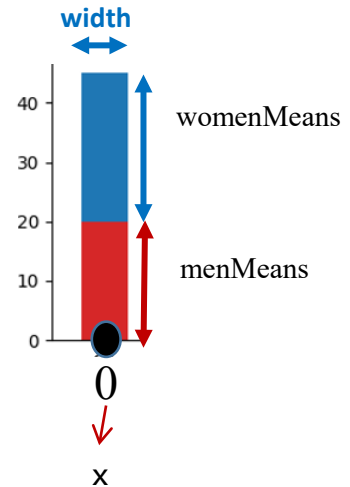
```
p1 = plt.bar(x, menMeans, width, color= 'red', label="Men")
p2 = plt.bar(x, womenMeans, width, bottom = menMeans, label="Women")
```

```
plt.ylabel('Scores')
plt.title('Scores by group and gender')
```

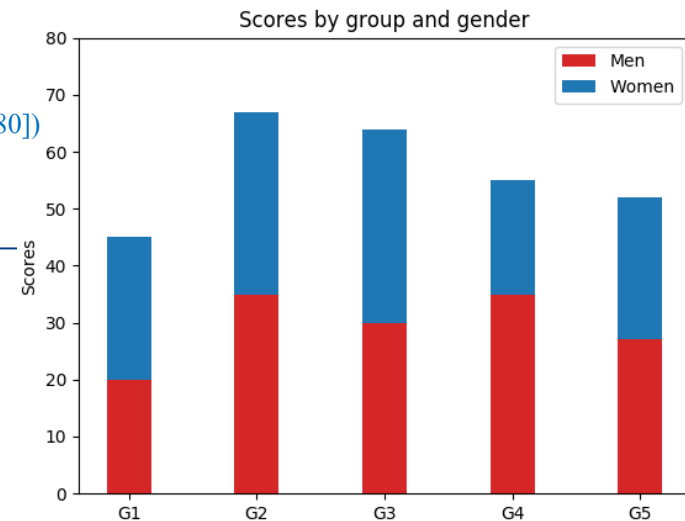
```
plt.xticks(x, ('G1', 'G2', 'G3', 'G4', 'G5'))
plt.yticks(np.arange(0, 81, 10))
```

← array([0,10,20,...,80])

```
plt.legend()
```



그림에서 womenMeans Bar chart
의 시작점: menMeans



Pyplot submodule functions

- plot() in PyPlot
- scatter() in PyPlot
- bar() in PyPlot
- pie() in PyPlot
- hist() in PyPlot
- boxplot() in PyPlot
- figure() & subplot() in PyPlot

- Matplotlib 작업
 - submodule pyplot 에 있는 함수 실행
 - 그림 그리는 대상 Object (Pyplot Function들의 input)
 - Python list or Numpy ndarray or Pandas Object

plt.pie() : Plotting Pie Chart

[1/3]

■ matplotlib.pyplot.pie(x, labels, colors, ...)

x: wedge size

labels: sequence of labels

colors: sequence of colors

shadow: draw shadow beneath pie

startangle: start pie chart with this degrees

autopct: label wedges with numeric value

```
from numpy import *
import matplotlib.pyplot as plt
names = ['Frogs', 'Hogs', 'Dogs', 'Logs']
p_size = [15, 30, 45, 10]

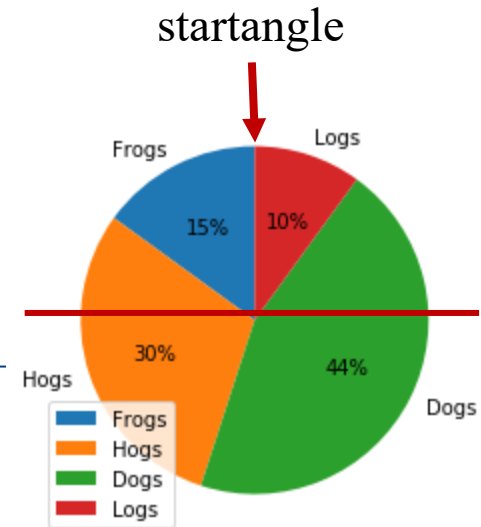
plt.pie(p_size, labels = names, autopct = '%d%%', startangle = 90)
plt.legend()
```

pie object의 piece (조각)
들에 배당된 labels의 값
들로 legend를 생성

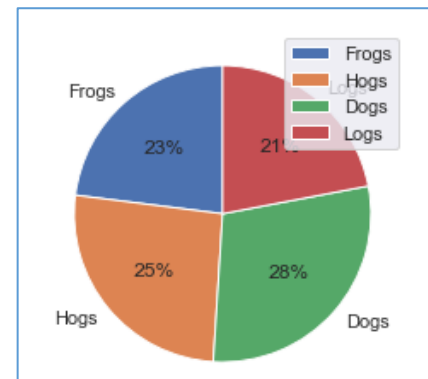
auto-percent: 자동으로
데이터들간에 percent 계산

%d : 정수
%% : % mark
%f : 실수
%.2f : 소수점 2자리까지의 실수
%0.2f : 소수점 2자리까지의 실수
%1.2f : 소수점 2자리까지의 실수

“colors” parameter 값을 안주면: Default color listing



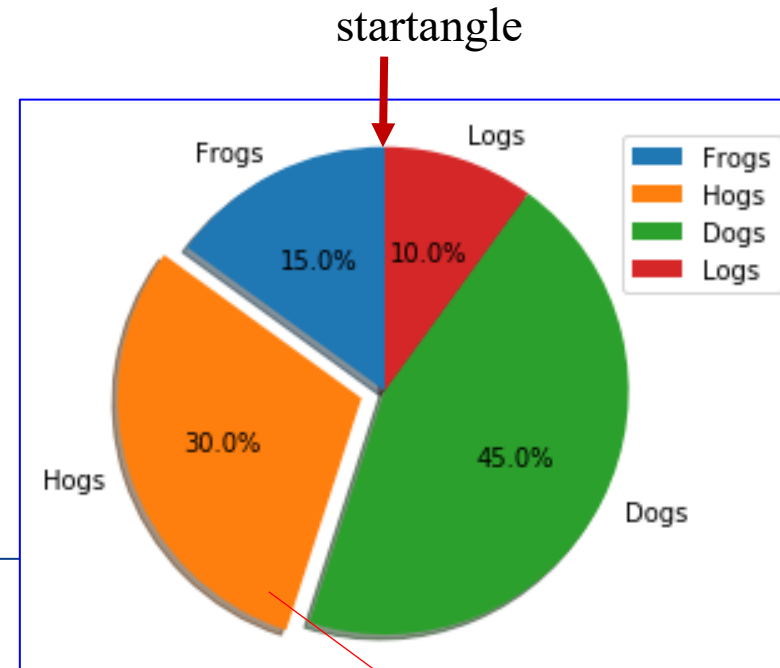
If sizes = [115, 130, 145, 110]



plt.pie() : Plotting Pie Chart

[2/3]

- explode parameter : 튀어나온 pie
- shadow parameter : pie 아래 그림자
- plt.axis('equal') : x-axis와 y-axis의 scale을 같게함



```
import matplotlib.pyplot as plt
```

```
labels = ['Frogs', 'Hogs', 'Dogs', 'Logs']
```

```
sizes = [15, 30, 45, 10]
```

```
explode = (0, 0.1, 0, 0) # only "explode" the 2nd slice
```

```
plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%',  
        shadow=True, startangle=90)
```

```
plt.axis('equal')
```

```
plt.legend()
```

(소수점 첫째자리 실수)%

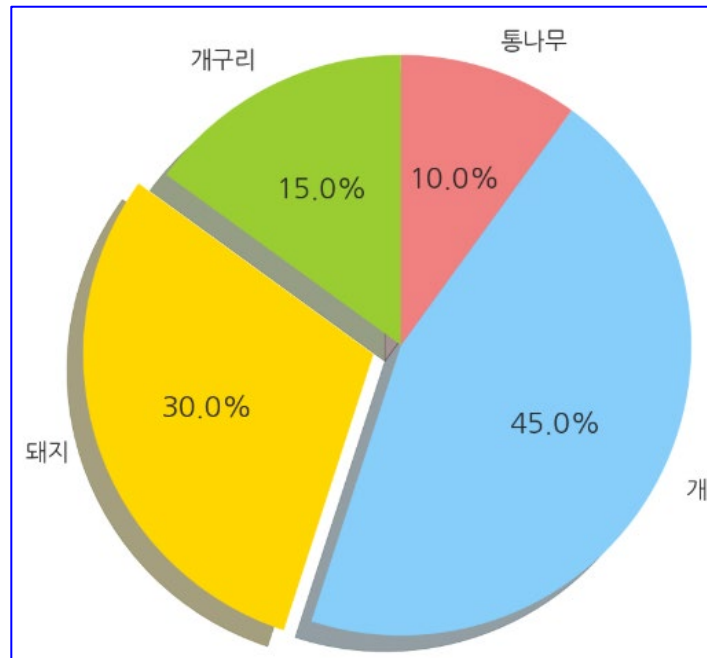
plt.pie() with Specific Color List

[3/3]

```
labels = ['개구리', '돼지', '개', '통나무']  
sizes = [15, 30, 45, 10]  
colors = ['yellowgreen', 'gold', 'lightskyblue', 'lightcoral']  
explode = (0, 0.1, 0, 0)
```

User-defined colors

```
plt.pie(sizes, explode=explode, labels=labels, colors=colors,  
autopct='%1.1f%%', shadow=True, startangle=90)  
plt.axis('equal')  
plt.show()
```



Pyplot submodule functions

- plot() in PyPlot
- scatter() in PyPlot
- bar() in PyPlot
- pie() in PyPlot
- hist() in PyPlot
- boxplot() in PyPlot
- figure() & subplot() in PyPlot

- Matplotlib 작업
 - submodule pyplot 에 있는 함수 실행
 - 그림 그리는 대상 Object (Pyplot Function들의 input)
 - Python list or Numpy ndarray or Pandas Object

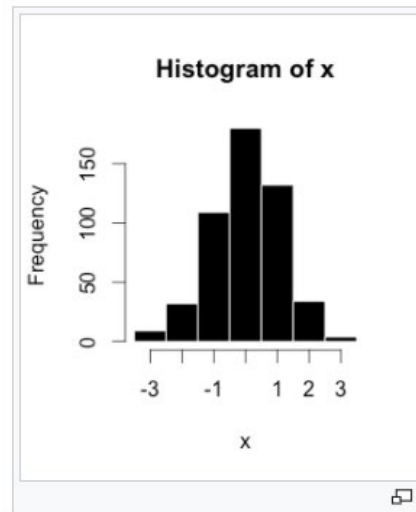
Histogram (도수분포)

■ Histogram의 예

Bin No	Range	Frequency
1	-3.5 ~ -2.5	23
2	-2.5 ~ -1.5	32
3	-1.5 ~ -0.5	109
4	-0.5 ~ 0.5	132
5	0.5 ~ 1.5	34
6	1.5 ~ 2.5	2
7	2.5 ~ 3.5	90

■ Histogram Graph

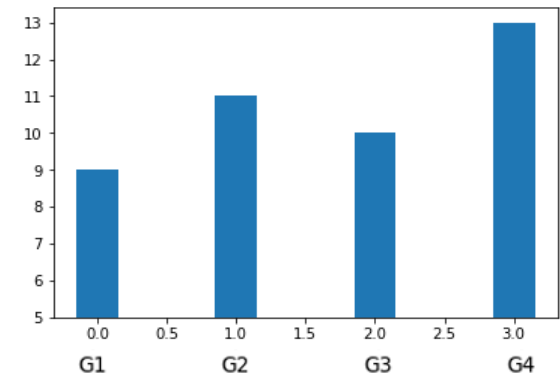
Histogram을 나타내는 방법



- X 축: Continuous Ranges of Numeric Data
- Histogram bars are touching

vs

Bar Chart



- X 축: Categorical Values
- Bar-chart bars are not touching

plt.hist() : Plotting Histogram Graph

[1/2]

- Generate 100 random floating numbers in range of 0. ~ 10.

```
v = np.random.uniform(0., 10., 100)
```

Uniform
Distribution



V

0~10의 random number 100개

```
array([9.59907953, 0.02422541, 2.82054237, 8.84408103, 4.52971878,  
       7.21823601, 4.30180311, 5.79395615, 5.56230367, 4.19906282,  
       0.4331126 , 0.42914631, 6.52259286, 0.55867994, 8.94271047,  
       ... ...])
```



- 0 ~ 10 범위의 random number 100개 이므로
 - $0.0 \leq x < 1.0$ 에 생성된 random number 가 몇 개일까?
 - $1.0 \leq x < 2.0$ 에 생성된 random number 가 몇 개일까?
 -
 - $9.0 \leq x < 10.0$ 에 생성된 random number 가 몇 개일까?

np.histogram(data, bins) : histogram 생성

```
v = np.random.uniform(0., 10., 100)
```

v

→ array([7.71917613, 5.46038868, 0.7056038 , 9.01156863, 2.34458166, 8.31034584, 9.27190639, 2.05601726, 0.03877716, 2.67049791, 1.80417496, 8.18565576, 6.08609183, 6.53024789, 7.9501905 , 5.91642226, 7.76078013, 7.59121856, 6.87032138, 7.86173525, 3.05182249, 7.34972919, 1.14114167, 8.88889549, 3.72122469, 3.06080852, 0.28104509, 5.5032323 , 3.49821743, 2.21449104, 8.746526 , 8.78523278, 9.5418974 , 2.56071025, 5.99846098, 8.7112315 , 8.34221145, 2.28728022, 9.6960325 , 2.69727739, 4.97247866, 2.59136536, 2.40095356, 6.9231556 , 4.44298028, 0.56629159, 2.79331772, 1.33284466, 4.38097902, 1.98368742, 3.32588233, 3.36582188, 3.68014307, 5.16384986, 7.2735741 , 4.63840068, 3.82631596, 7.64291183, 6.26431395, 8.66966622, 5.25901477, 7.93373297, 8.703527 , 4.68545306, 0.58719662, 7.68737052, 2.29028986, 8.28843443, 2.19210156, 3.06808559, 8.07512003, 7.3252879 , 5.22408342, 7.39561824, 5.67726218, 6.82559876, 8.54477967, 0.73807491, 8.0280058 , 4.96838193, 8.9794383, 3.9727384, 0.77425737, 0.13373396, 8.46870522, 3.50336129, 8.80442185, 8.28282849, 0.83415309, 2.97980913, 5.15114653, 9.18546466, 7.89065919, 6.78565558, 9.64448022, 2.03647588, 1.09179448, 9.44118404, 4.23874048, 0.52651302])

```
histogram_data , bins_made = np.histogram( v, bins= np.arange(0, 11) )
```

histogram_data

→ array([11, 14, 9, 7, 6, 12, 9, 11, 10, 11], dtype=int64)

bins_made

→ array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

	bins_made	histogram_data
1	0 ~ 1	11
2	1 ~ 2	14
3	2 ~ 3	9
....	...	
7	6 ~ 7	9
8	7 ~ 8	11
9	8 ~ 9	10
10	9 ~ 10	11

plt.hist() : Plotting Histogram Graph

■ matplotlib.pyplot.hist(x, range, bins, ...)

x: input data

range: lowest and highest number of x axis
default – (min, max) of dataset

bins: number of bins (default = 10)

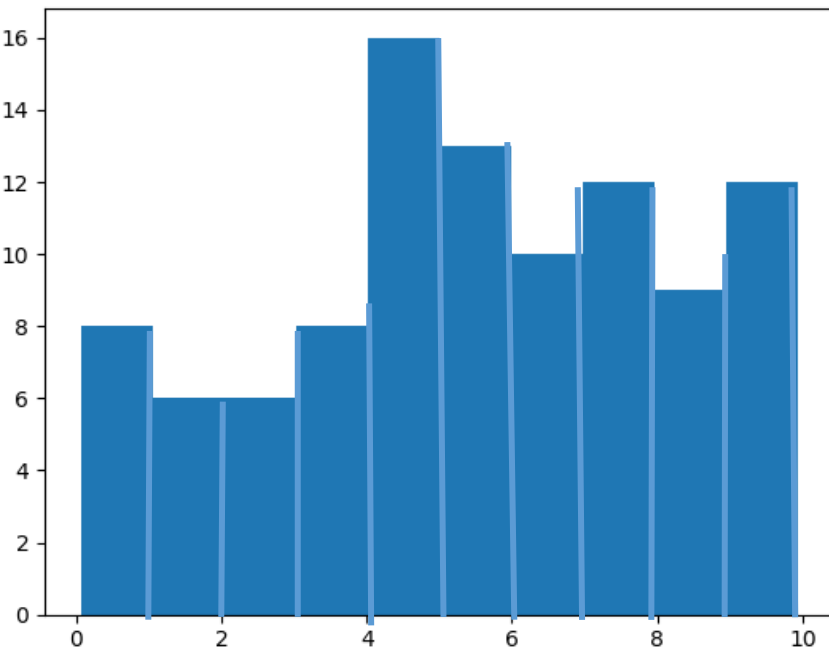
```
v = np.random.uniform(0., 10., 100)
```

V → 0~10의 random number 100개

```
array([9.59907953, 0.02422541, 2.82054237, 8.84408103, 4.52971878,
       7.21823601, 4.30180311, 5.79395615, 5.56230367, 4.19906282,
       0.4331126 , 0.42914631, 6.52259286, 0.55867994, 8.94271047,
       ... ...])
```

```
plt.hist(v)
```

- V의 범위가 0 ~ 10 이므로 10 bins 생성
 - Bin1 has the count of $0.0 \leq x < 1.0$
 - Bin2 has the count of $1.0 \leq x < 2.0$
 -
 - Bin10 has the count of $9.0 \leq x < 10.0$



- 주어진 Data의 range를 보고 10개의 Bin을 만들
- 10 개 Bin에 100개를 배당하다 보니, 어떤 Bin에는16개도 배당

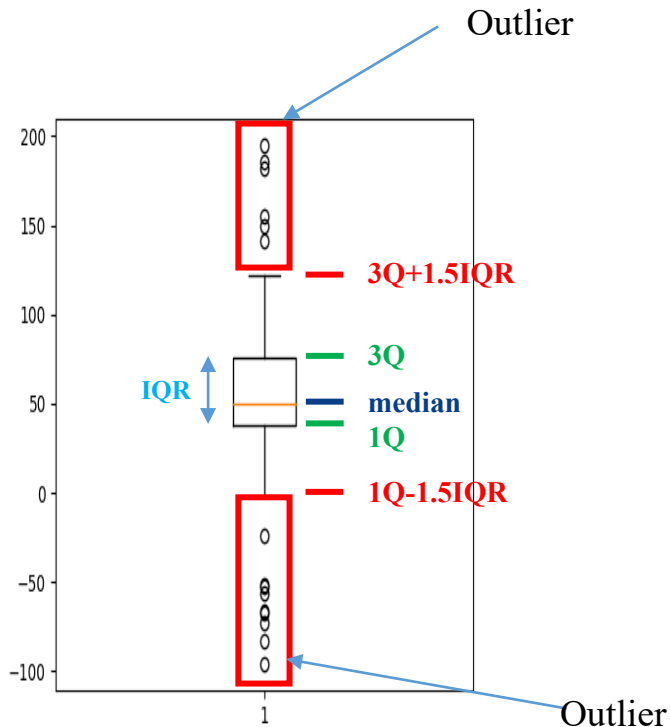
Pyplot submodule functions

- plot() in PyPlot
- scatter() in PyPlot
- bar() in PyPlot
- pie() in PyPlot
- hist() in PyPlot
- boxplot() in PyPlot
- figure() & subplot() in PyPlot

- Matplotlib 작업
 - submodule pyplot 에 있는 함수 실행
 - 그림 그리는 대상 Object (Pyplot Function들의 input)
 - Python list or Numpy ndarray or Pandas Object

Box Plot (or Box-and-Whisker Plot)

- Mary Eleanor Spear, 1952년, <Charting Statistics> : "범위 막대(range bar)" 제안
- John Tukey, 1977년, "탐색적 데이터 분석(Exploratory Data Analysis)" 에서 소개
- 다섯가지 기초 통계량: 최소값, 제1사분위수, 제2사분위수, 제3사분위수, 최대값
 - '5가지 요약 수치(five-number summary)'



IQR: Inter Quartile Range (사분위 범위)
= 3사분위의 수 - 1사분위의 수
= 75% 지역의 값 - 25% 지역의 값
= $3Q - 1Q$

Outlier X :

$X > 3Q + 1.5 * IQR$ 혹은 $X < 1Q - 1.5 * IQR$

plt.boxplot() : Box Chart 그리기

[1/6]

- Boxplot을 잘 설명하기 위한 데이터 생성

```
import matplotlib.pyplot as plt
import numpy as np
```

```
# fake up some data
```

```
spread = np.random.rand(50) * 100
```

```
center = np.ones(25) * 50
```

0~100 범위 임의의 실수 50개 생성

50을 25개 생성 → 50을 median 값으로 하려고

```
filter_high = np.random.rand(10) * 100 + 100
```

```
filter_low = np.random.rand(10) * (-100)
```

Outlier: 100 ~ 200 범위의 실수 10개
Outlier: -100 ~ 0 범위의 실수 10개

```
data = np.concatenate((spread, center, filter_high, filter_low), 0)
```

```
# basic plot
```

```
plt.boxplot(data)
```

총 95개 숫자

axis = 0 : row로 추가

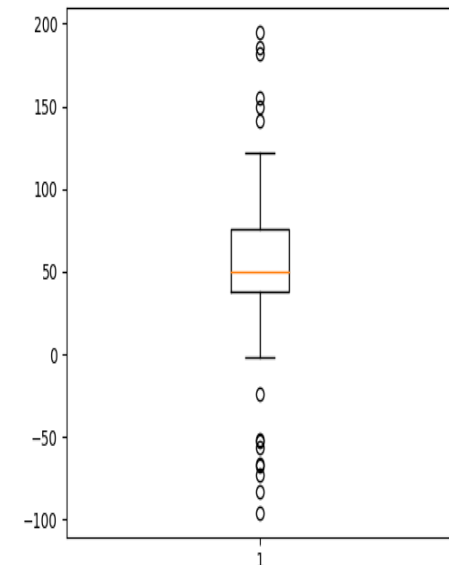
■ matplotlib.pyplot.boxplot(x, notch, sym, vert, ...)

x: input data

notch (0): **True(1)** – notched, **False(0)** – rectangle

sym (**None**): shape of outliers symbols, **None**-black circle

vert (**1**): **True(1)** – vertical, **False(0)** – horizontal



Numpy concatenate()

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
np.concatenate((a, b), axis=0)
```

axis가 0 이면 row로 추가

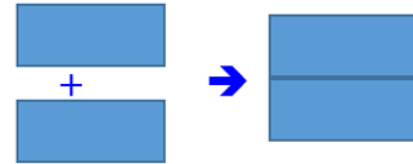
→ array([1, 2],
[3, 4],
[5, 6]))

[1, 2]
[3, 4]
+
[5, 6] → [1, 2]
[3, 4]
[5, 6]

```
np.concatenate((a, b.T), axis=1)
```

→ array([[1, 2, 5],
[3, 4, 6]])

vstacking() : vertical stacking



axis가 1 이면 column으로 추가

[1, 2] + [5]
[3, 4] [6] → [1, 2, 5]
[3, 4, 6]

hstack() : horizontal stacking



plt.boxplot() : Box Chart 그리기 [2/6]

■ Notched boxplot

```
spread = np.random.rand(50) * 100
center = np.ones(25) * 50
filter_high = np.random.rand(10) * 100 + 100
filter_low = np.random.rand(10) * (-100)
data = np.concatenate((spread, center, filter_high, filter_low), 0)
```

```
# notched plot
```

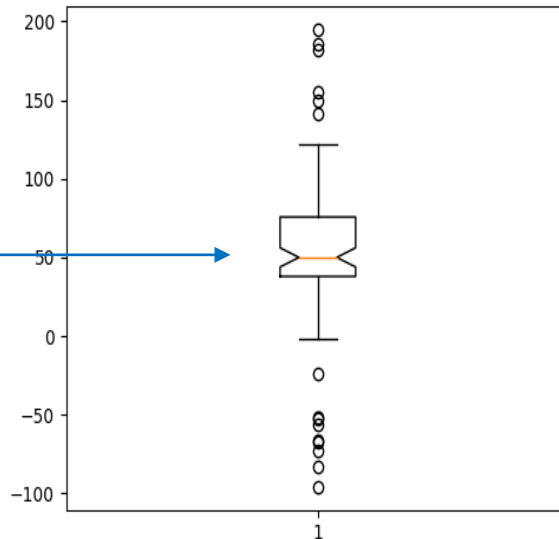
```
plt.figure()
```

```
plt.boxplot(data, notch = 1)
```

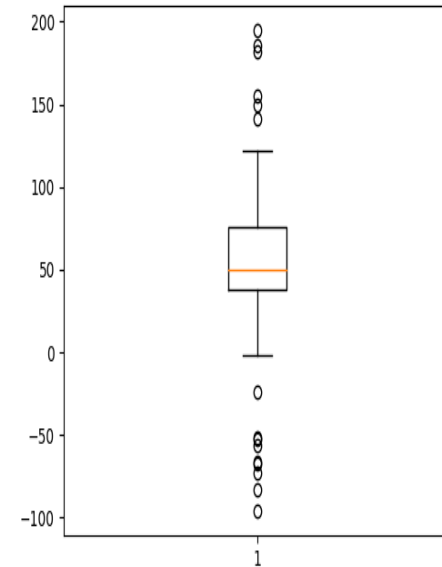
Figure() 는 window 그리는 기능, default로 1개 figure setting

notched box

주어진 데이터의
median 값을 강조



Plain BoxPlot



Default outlier symbol: black circle

np.boxplot() : Box Chart 그리기 [3/6]

■ Change outliers' symbol

```
spread = np.random.rand(50) * 100
center = np.ones(25) * 50
filter_high = np.random.rand(10) * 100 + 100
filter_low = np.random.rand(10) * (-100)

data = np.concatenate((spread, center, filter_high, filter_low), 0)

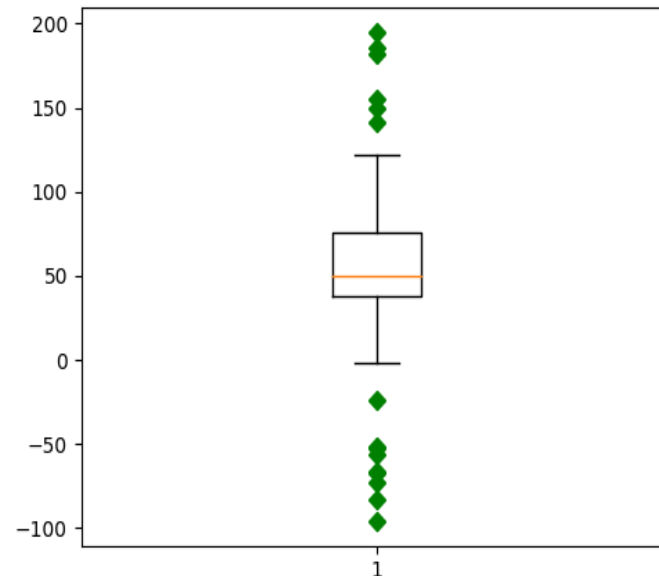
# change outlier point symbols
plt.figure()
plt.boxplot(data, notch = 0, sym = 'gD')
```

regular box
(Default)

outlier symbol

* gD: green diamond

- matplotlib의 boxplot은 legend를 추가하지 못하고, 나중에 seaborn의 boxplot에서는 legend 추가가 가능



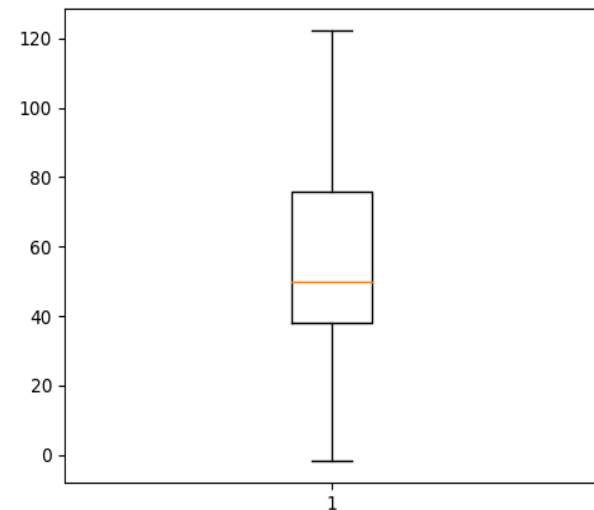
plt.boxplot() : Outliers 없이 Plotting Box Chart 그리기 [4/6]

```
spread = np.random.rand(50) * 100
center = np.ones(25) * 50
filter_high = np.random.rand(10) * 100 + 100
filter_low = np.random.rand(10) * (-100)
data = np.concatenate((spread, center, filter_high, filter_low), 0)

# ignore outliers
plt.figure()
plt.boxplot(data, notch = 0, sym = '')
```

regular box
(Default)

Display No Outliers



plt.boxplot() : Horizontal Box Chart 그리기

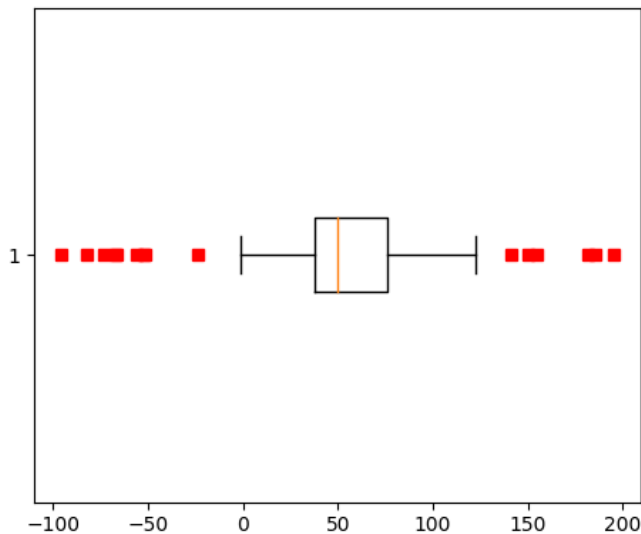
[5/6]

```
spread = np.random.rand(50) * 100
center = np.ones(25) * 50
filter_high = np.random.rand(10) * 100 + 100
filter_low = np.random.rand(10) * (-100)
data = np.concatenate((spread, center, filter_high, filter_low), 0)

# horizontal boxes
plt.figure()
plt.boxplot(data, notch = 0, sym= 'rs', vert = 0)
```

* rs: red square

0: horizontal box
1: vertical box (default value)



plt.boxplot() : Multi Box Chart 그리기 [6/6]

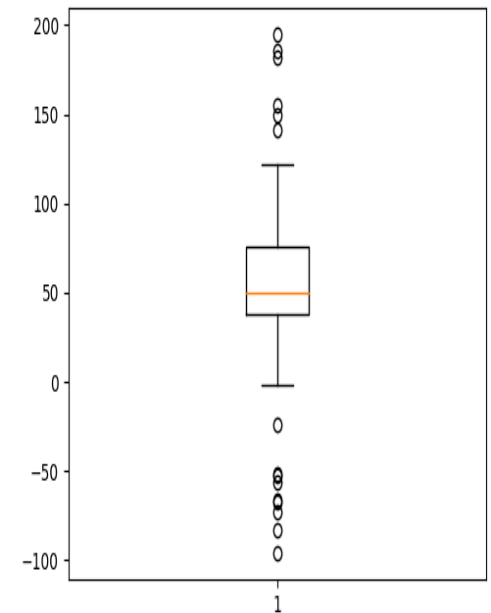
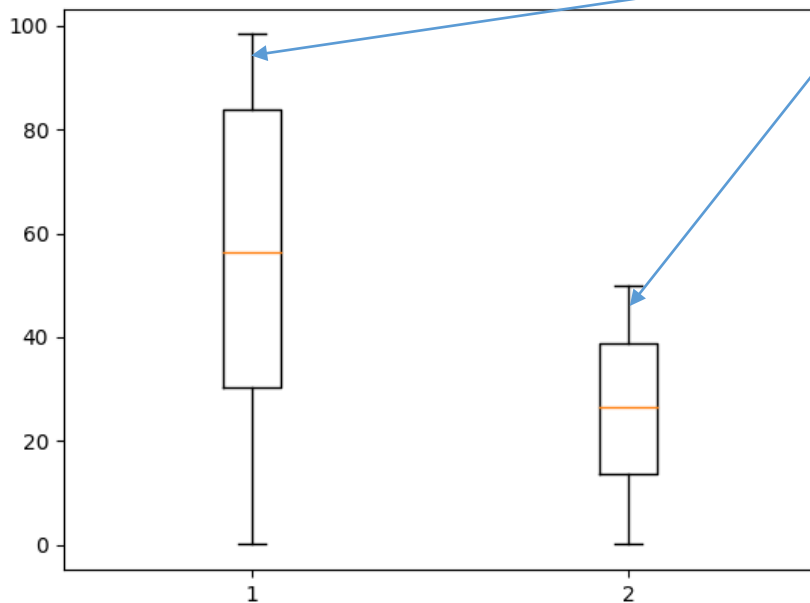
List안에 여러개의 numpy array가 있으면 multiple box chart들을 그린다

```
# multiple boxplots on one figure
plt.figure()
data = [ np.random.rand(50) * 100, np.random.rand(50) * 50 ]

plt.boxplot(data)
```

No Outliers! : 모든 생성된 실수가 0 ~ 100 or 0 ~ 50 범위에 있다

- $3Q + 1.5 * IQR$ 보다 큰값이나 $1Q - 1.5 * IQR$ 보다 작은값의 생성이 없음



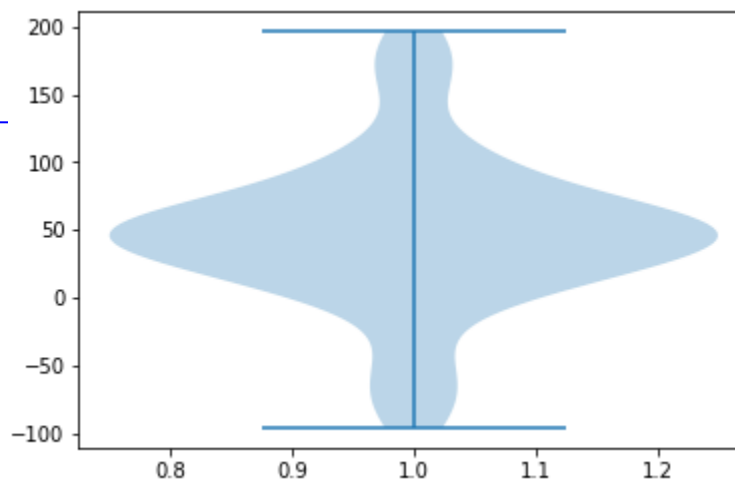
Box Plot Default Values: Vertical, No-Notch, Black Circle Outlier

plt.violinplot(): box chart에 분포를 표현

Box대신 Violin모양으로 데이터의 분포를 볼수 있는 violinplot()

```
spread = np.random.rand(50) * 100
center = np.ones(25) * 50
filter_high = np.random.rand(10) * 100 + 100
filter_low = np.random.rand(10) * (-100)
data = np.concatenate((spread, center, filter_high, filter_low), 0)

# horizontal violin shape
plt.figure()
plt.violinplot(data)
```



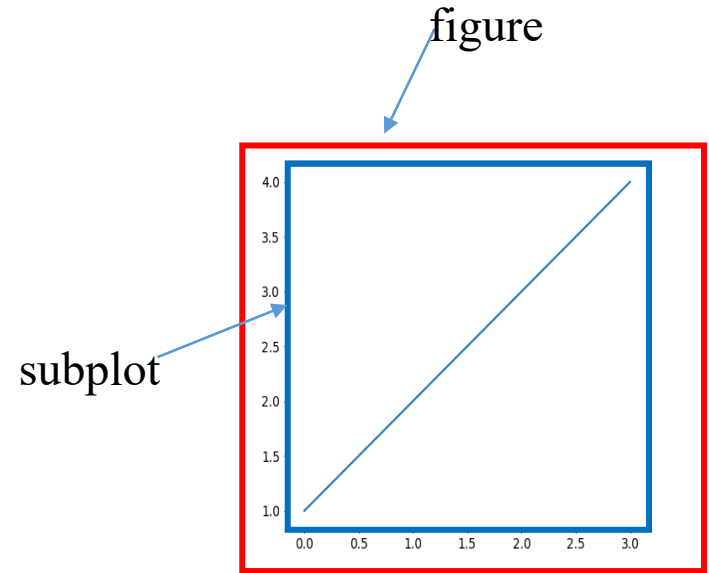
Pyplot submodule functions

- plot() in PyPlot
- scatter() in PyPlot
- bar() in PyPlot
- pie() in PyPlot
- hist() in PyPlot
- boxplot() in PyPlot
- figure() & subplot() in PyPlot

- Matplotlib 작업
 - submodule pyplot 에 있는 함수 실행
 - 그림 그리는 대상 Object (Pyplot Function들의 input)
 - Python list or Numpy ndarray or Pandas Object

figure() and subplot() in matplotlib

```
import matplotlib.pyplot as plt  
plt.plot([1, 2, 3, 4])
```



```
import matplotlib.pyplot as plt  
plt.figure( )  
plt.subplot(111)  
plt.plot([1, 2, 3, 4])
```



1 figure, 1 subplot 이면 plt.figure() 와 plt.subplot()은 생략이 가능

plt.figure(*d*): matplotlib이 그래프를 그릴 때, 이미 생성된 figure 창이 있으면 그 창을 사용하고, 없다면 새 figure를 자동으로 만들어 사용한다

plt.subplot(*d₁d₂d₃*): matplotlib이 그래프를 그릴 때, 이미 만들어진 subplot(부분 그래프 창)이 있으면 그것을 사용하고, 없으면 새로운 subplot을 자동으로 만든다

plt.figure() and plt.subplot()

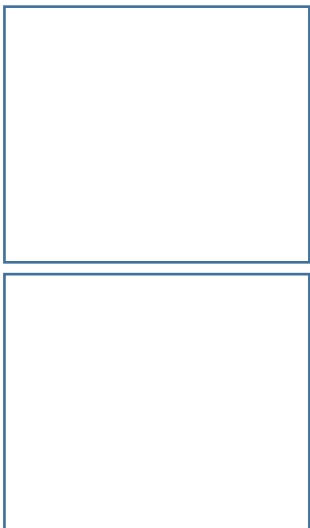
plt.figure()

→



plt.figure(1)
plt.figure(2)

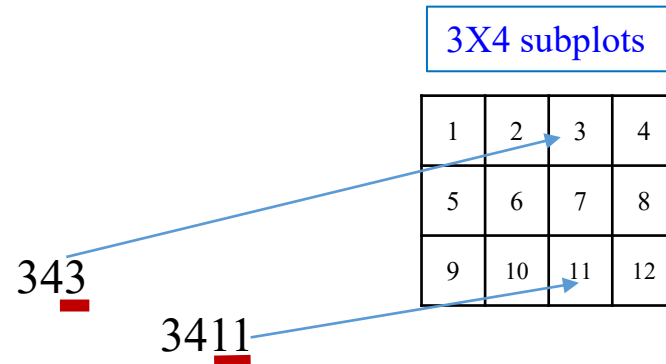
→



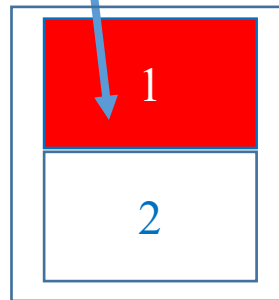
subplot($d_1 d_2 d_3$)

$d_1 = \text{rows}$, $d_2 = \text{columns}$, $d_3 = \text{subplot_num}$

If $d_1 = 3$, $d_2 = 4$, then d_3 will be mapped as below

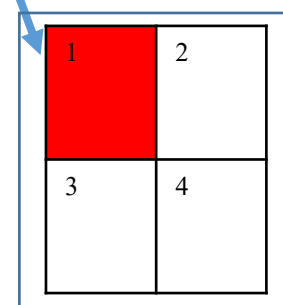


subplot(211)



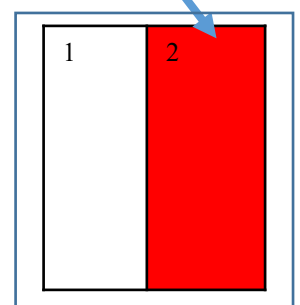
2 x 1 subplots

subplot(221)



2 x 2 subplots

subplot(122)



1 x 2 subplots

plt.figure() and plt.subplot()

[1/5]

```
import numpy as np
import matplotlib.pyplot as plt
```

$$f(t) = e^{-t} \cos(2\pi * t)$$

```
def f(t):
    return np.exp(-t) * np.cos(2 * np.pi * t)
```

```
t1 = np.arange(0.0, 5.0, 0.1)  ← 50개의 points
t2 = np.arange(0.0, 5.0, 0.02) ← 250개의 points
```

```
plt.figure(1)
plt.subplot(211)
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')
```

Blue Circle Marker

Black Color

$$e^{-t_1} \cos(2\pi * t_1)$$
$$e^{-t_2} \cos(2\pi * t_2)$$

```
plt.subplot(212)
plt.plot(t2, np.cos(2 * np.pi * t2), 'r--')
```

Red -- Marker

$$f(t_2) = \cos(2\pi * t_2)$$

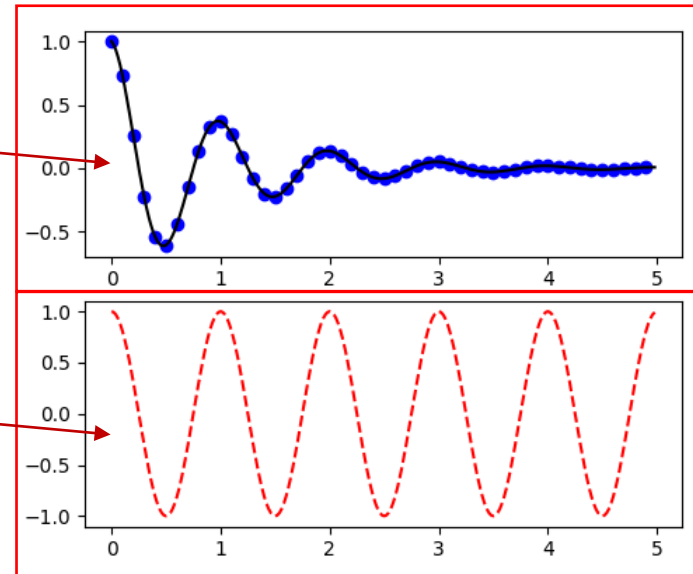
2 x 1 subplots

subplot(211)

1

subplot(212)

2



[2/5]

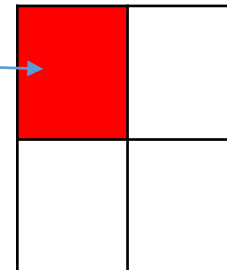
plt.figure() and plt.subplot()

```
plt.figure(1)
plt.subplot(221)
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')

plt.subplot(122)
plt.plot(t2, np.cos(2 * np.pi * t2), 'r--')
```

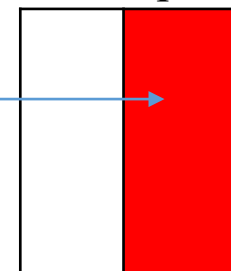
2 x2 subplots

subplot(221)

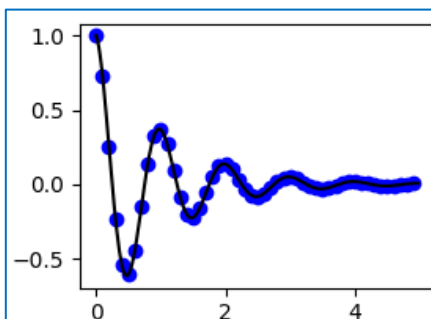


1 x2 subplots

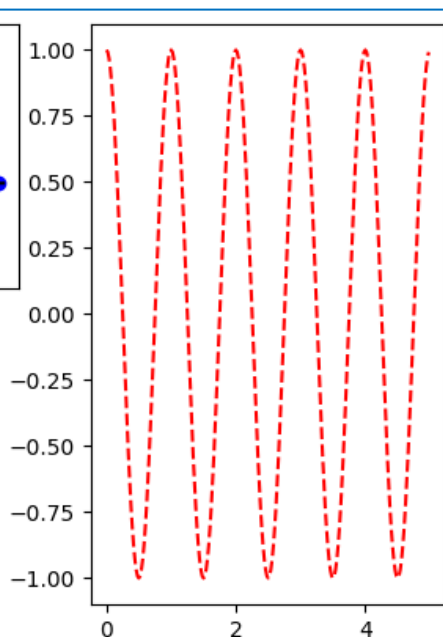
subplot(122)



$$f(x) = e^{-x} \cos(2 \pi x)$$



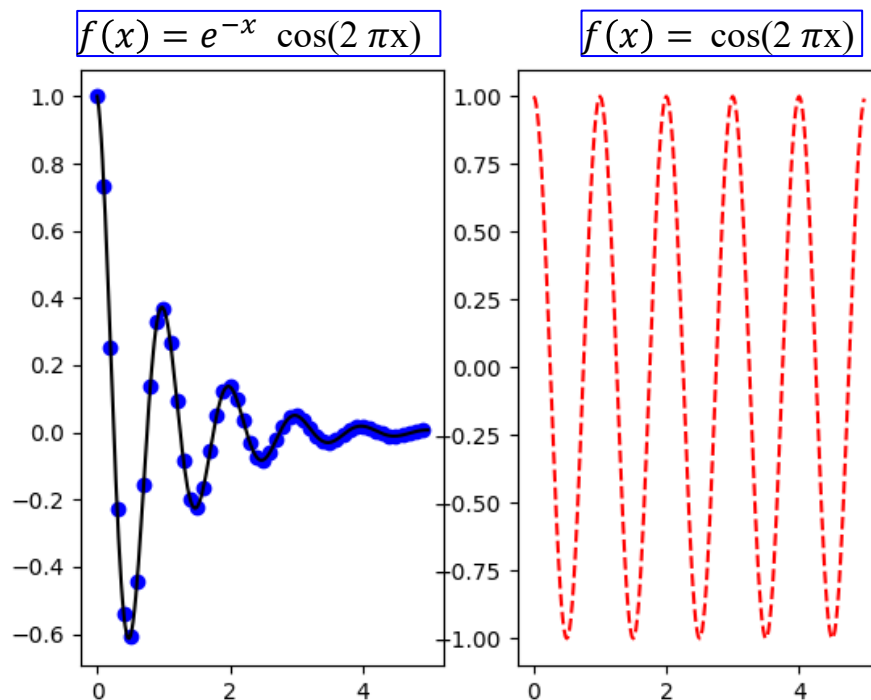
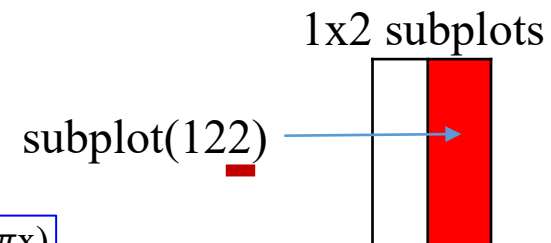
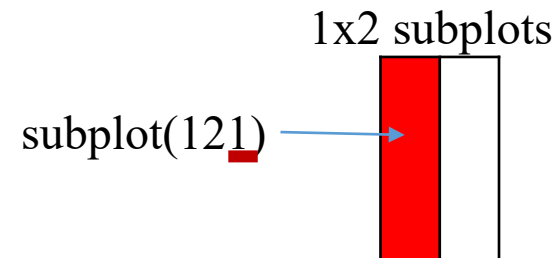
$$f(x) = \cos(2 \pi x)$$



plt.figure() and plt.subplot()

```
plt.figure(1)
plt.subplot(121)
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')

plt.subplot(122)
plt.plot(t2, np.cos(2 * np.pi * t2), 'r--')
```



plt.figure() and plt.subplot()

```
import matplotlib.pyplot as plt
```

code A

```
plt.figure(1)           # the figure 1 is created
plt.subplot(211)        # the subplot(211) is created
plt.plot([1, 2, 3])
plt.subplot(212)        # the subplot(212) is created
plt.plot([4, 5, 6])

plt.figure(2)           # the figure 2 is created
plt.plot([4, 5, 6])      # the subplot(111) is created by default
```

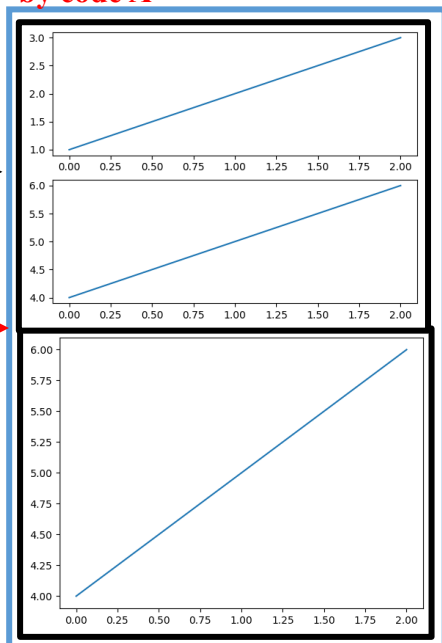
Code A가 수행된 후에 figure 2와 subplot(212)에 current pointer가 있는 상태

code B

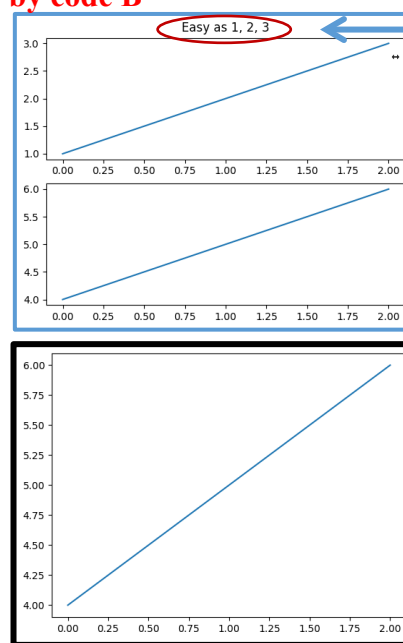
```
plt.figure(1)
plt.subplot(211)
plt.title('Easy as 1, 2, 3')
```

Code B에서 current pointer를 figure 1과 subplot(211)로 변경을 시킨다

by code A



by code B



figure(1)'s subplot(211)에서 plt.title() 수행!



plt.figure() and plt.subplot()

[5/5]

Self study

```
import matplotlib.pyplot as plt
```

Code 1

```
plt.figure(1)
Plt.subplot(211)
Plt.plot([1, 2, 3])
Plt.subplot(212)
Plt.plot([4, 5, 6])
```

```
plt.figure(2)
plt.plot([4, 5, 6])
```

Code 1이 끝난순간에 current pointer가 figure 2를 pointing

Code 2

```
plt.clf()
```

마지막에 접근했던 figure 2 제거

```
plt.figure(1)
```

figure 1으로 다시 오면 마지막에 접근했던 subplot이 subplot (212) 이였었다!

```
plt.cla()
```

```
plt.subplot(211)
```

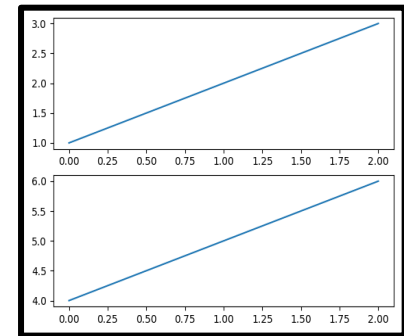
```
plt.title('Easy as 1, 2, 3')
```

clf(): clear current Figure
마지막에 접근한 figure를 제거

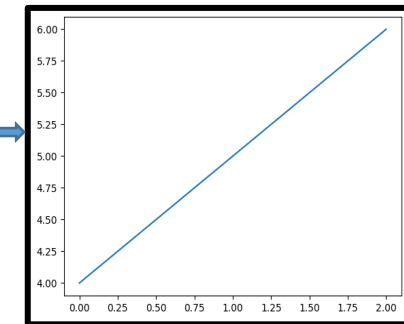
cla(): clear current Axes
마지막에 접근한 subplot의 내용물을 제거

지금 current pointer가 pointing하는 figure 1의 subplot(212)의 내용물을 제거

figure(1)



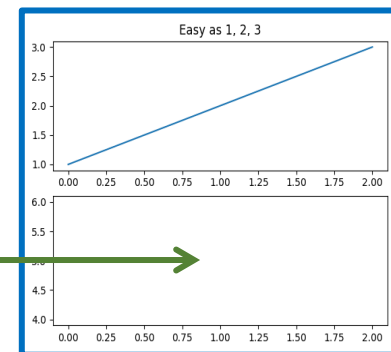
figure(2)



by code 1

by code 2

figure(1)



AI Coding 을 위해 알아야할 Matplotlib의 detail 들

- **Figure vs Axes (구조의 이해):** **Figure**는 전체 도화지, **Axes**는 그 안의 개별 그래프(축 포함)를 의미하며, 한 화면에 여러 그래프를 배치
- **Pyplot submodule의 기본함수이해:** `plot()` / `scatter()` / `bar()` / `pie()` / `hist()` / `boxplot()` / `figure()` / `subplot()`
- **Subplots Layout (서브플롯 배치):** `plt.subplot()`이나 `GridSpec`을 사용해 여러 모델의 성능 지표(Loss, Accuracy)를 한눈에 비교할 수 있도록 배치
- **Colormaps (컬러맵 활용):** Heatmap이나 가중치 시각화시 데이터의 특성에 따라 `viridis`(순차적)등 적절한 컬러맵을 선택
- **Legend & Labels (가독성):** `label()`, `title()`, `티뮤디()`, `ylabel()`은 기본이며, `loc='best'` 옵션으로 범례가 데이터 흐름을 가리지 않게 자동 배치